R.M.K. ENGINEERING COLLEGE

(An Autonomous Institution)

R.S.M Nagar, Kavaraipettai, Gummidipoondi Taluk, Thiruvallur District, Tamil Nadu-601206 Affiliated to Anna University, Chennai / Approved by AICTE, New Delhi
Accredited by NAAC with A+ Grade /An ISO 21001:2018 Certified Institution All the Eligible UG Programs are Accredited by NBA, New Delhi

24ME511- PRODUCT DEVELOPMENT LAB – III

REPORT ON

MOVIEMIND: AI-POWERED MOVIE RECOMMENDATION SYSTEM

Submitted By

Vikranath A – 111724203119

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

R.M.K ENGINEERING COLLEGE

(An Autonomous Institution)

R.S.M Nagar, Kavaraipettai - 601 206

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this Product Development Lab III report "MovieMind: AI-Powered Movie Recommendation System" is the bonafide work of **“ Vikranath A (111724203119) ”** who carried out the 24ME511 – Product Development Lab III under my supervision.

SIGNATURE
Dr. Sheerin Banu M, M.Tech, Ph.D

Professor and Head,

Dept of Information Technology,

R.M.K. Engineering College,

R.S.M. Nagar,

Kavarapettai - 601206.

SIGNATURE
Ms. S. Nandhini,

Supervisor, Assistant Professor,

Dept of Information Technology,

R.M.K. Engineering College,

R.S.M Nagar,

Kavarapettai - 601206.

CERTIFICATE OF EVALUATION

College Name : R.M.K. ENGINEERING COLLEGE

Department : INFORMATION TECHNOLOGY

Semester : 05

Title of Project	Name of the Students	Name of the Supervisor with designation
MovieMind: AI-Powered Movie Recommendation System	Vikranath A (111724203119)	Ms. S. Nandhini, Assistant Professor

The report of the Product Development work submitted by the above students in partial fulfilment for the award of Bachelor of Technology Degree in **INFORMATION TECHNOLOGY** of Anna University was evaluated and confirmed to be the report of the work done by the above students and then evaluated.

Submitted for the project viva-voce examination held on .....

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We earnestly portray our sincere gratitude and regards to our beloved Founder Chairman, **Thiru. R. S. MUNIRATHINAM** for giving us the infrastructure for conducting the project work and our Chairperson **Tmt. MANJULA MUNIRATHINAM** for her blessings. We also thank our Vice Chairman, **Thiru. R. M. KISHORE** and our Director, **Thiru. R. JOTHI NAIDU** for their constant support and affection shown towards us throughout the course.

We are extremely thankful to our Principal, **Dr. K. A. MOHAMED JUNAID**, for being the source of inspiration in this college.

We reveal our sincere thanks to our Professor and Head of the Department, Information Technology, **Dr. M. SHEERIN BANU**, for her commendable support and encouragement for the completion of our project.

We would like to express our sincere gratitude to our Course Coordinator **Dr. [Course Coordinator name]**, for her/his valuable suggestions towards the successful completion of this project.

We wish to record our thanks to our project supervisor **Ms. S. Nandhini**, for her valuable guidance and support during each stage of our project.

We take this opportunity to extend our thanks to all the faculty members of the Department of Information Technology, our parents and friends for their care and support towards the completion of our project.

ABSTRACT

Choosing what to watch is a problem of abundance. Streaming catalogues hold tens of thousands of titles, yet most recommender systems describe a viewer with a fixed profile, such as a list of liked genres or a row of past ratings, and treat the viewer's current mood either as a hard genre filter or not at all. Genre labels are too coarse to describe what a film feels like, keyword-based text matching such as TF-IDF cannot connect synonyms, and a mood filter that ignores taste can return films the viewer would never enjoy.

This project presents MovieMind, a web-based movie recommendation system built around one idea: movies, users and moods can live in the same mathematical space. Each movie is converted into a 384-dimensional vector by the all-MiniLM-L6-v2 Sentence Transformer applied to its plot, tagline, genres, keywords, director and cast. A user is represented by the centroid of the vectors of the movies they like. Each of ten moods is represented by a direction vector computed as the mean of a set of exemplar movies minus the mean of all movies. A recommendation is the set of movies nearest to the search position user_taste + λ × mood_direction, where λ is a number between 0 and 1 that the viewer controls with a slider.

Because the user, the movies and the mood share one space, the system also explains each recommendation from the geometry: how well it matches the user's taste, how much the mood shift changed its ranking, and which previously liked movie it is closest to. The system is implemented with Flask, SQLite, NumPy and scikit-learn, runs on a CPU with no paid API, and adds a single dependency, sentence-transformers, to the earlier TF-IDF version of the project.

The final design is a hybrid. It blends content similarity with a collaborative signal (what people with similar likes enjoyed) and a quality prior into a base score, retrieves the 60 best unseen films, and then lets the chosen mood re-rank them by the strength λ, so the mood changes the order without lowering match quality. The approach was evaluated offline on MovieLens with users held out: item statistics come from 60% of users, blend weights are tuned on 20% and results are reported on the remaining 20%, over five random splits. Hit Rate@10 is 0.44 for the hybrid, against 0.15 for the earlier TF-IDF engine, 0.13 for content embeddings alone and 0.35 for recommending the most popular films, and the hybrid is ahead of popularity in every split. The first design, which moved the search point toward the mood, trailed popularity; the redesign fixed this. The mood re-rank changes about two of ten titles at the default strength without a measurable loss in accuracy, and with an exactly known mood it improves hit rate, but

the offline data contain no mood labels, so whether a user-chosen mood improves satisfaction remains untested.

The system was implemented end to end with 99 automated tests, answers a request in a median of 3.6 ms on a CPU, and has a redesigned interface with a mood slider and a taste summary. The report documents the design, both evaluations, the limitations (small dataset, no user study, noisy weight selection) and a roadmap that includes stronger collaborative models, multi-interest taste and a user study.

Keywords: Recommender System, Sentence Embeddings, Semantic Search, Mood-Aware Recommendation, Explainable AI, Content-Based Filtering, Cosine Similarity, Flask, MovieLens.

TABLE OF CONTENTS

Ch. No.	Title	Page No.
	Acknowledgement	4
	Abstract	5
	List of Figures	9
	List of Tables	10
	List of Abbreviations	12
1	INTRODUCTION	13
	1.1 Overview	13
	1.2 Scope of the Project	14
	1.3 Objectives	14
	1.4 Existing System	15
	1.5 Existing System Disadvantages	16
	1.6 Literature Review	16
	1.7 Proposed System	19
	1.8 Proposed System Advantages	20
	1.9 Hardware and Software Requirements	21
	1.10 Non-Functional Requirements	22
	1.11 Comparison with Alternative Approaches	22
2	SYSTEM DESIGN	24
	2.1 General Description	24
	2.2 Prototype / Block Diagram	24
	2.3 System Architecture (Data Flow Diagram)	24
	2.4 Use Case Diagram	25
	2.5 Activity Diagram (Algorithm Flowchart)	26
	2.6 UML Collaboration Diagram	27
	2.7 UML Class Diagram	28
	2.8 Database Design	29
	2.9 API Design	30
3	SYSTEM IMPLEMENTATION	31
	3.1 Module Description	31
	3.2 Module Interaction Summary	34
	3.3 Testing Strategy	34
	3.4 Deployment and Installation Procedure	35
4	TECHNOLOGY STACK	37
	4.1 Software Components	37
	4.2 Software Tools	39
	4.3 Communication Protocol Stack	39
	4.4 Resource and Cost Estimation	40
	4.5 Comparison with Alternative Embedding Choices	40
	4.6 Privacy, Security and Reliability	41
	4.7 Scalability	41
	4.8 Overall Technical Justification	41
5	SCREENSHOTS	42

5.1 System Configuration 42
5.2 Application Screenshots 42
5.3 Console Output 44
5.4 System Response and Validation 44
6 SAMPLE CODING 45
6.1 Embeddings and Mood Vectors 45
6.2 Embedding Store 45
6.3 Statistics and Hybrid Ranking 46
6.4 User Taste and Recommendation Engine 47
6.5 Explanations 48
6.6 REST Endpoint 49
6.7 API Quick Reference 50
7 RESULT AND DISCUSSION 51
7.1 Initial Experimental Setup (v2.0) 51
7.2 Initial Result 52
7.3 Initial Performance Metrics 52
7.4 Discussion of the Initial Evaluation 54
7.5 Design Change and Hybrid Evaluation Protocol 55
7.6 Hybrid Results 56
7.7 Discussion of the Hybrid Results 59
7.8 Catalogue Connectivity and Coverage Audit 60
7.9 Limitations of the Current Prototype 61
8 CONCLUSION AND FUTURE ENHANCEMENT 62
8.1 Conclusion 62
8.2 Applications 62
8.3 Future Enhancement 63
8.4 Societal and Ethical Considerations 64
8.5 Deployment Considerations 64
9 REFERENCES 65
Patent / Paper Publication Proof 66

LIST OF FIGURES

Figure No.	Description	Page No.
1.1	Schematic Semantic Map of Movies, User Taste and Mood (illustrative)	20
2.1	System Block Diagram	24
2.2	Data Flow / System Architecture Diagram	25
2.3	Use Case Diagram	26
2.4	Activity Diagram - Hybrid Recommendation with Mood Re-ranking	27
2.5	UML Collaboration Diagram	28
2.6	UML Class Diagram	29
3.1	Computing a Mood Direction Vector (schematic)	32
3.2	Hybrid Ranking Pipeline	33
4.1	Layered Software Architecture	37
4.2	Movie Encoding Pipeline	38
5.1	Login and Sign-up Page [placeholder]	43
5.2	Dashboard with Mood Selector and λ Slider [placeholder]	43
5.3	Recommendations with Geometric Explanations [placeholder]	44
7.1	HR@10 and NDCG@10 versus λ for proxy, random and oracle moods (shaded: 95% CI)	54
7.2	Comparison of Baselines and Proposed Method (95% bootstrap CI)	54
7.3	Hit Rate@10 and NDCG@10 of the Hybrid Variants on Held-Out Test Users (95% CI)	58
7.4	Effect of the Mood Re-rank Strength λ on Accuracy and on How Much the List Changes	58

LIST OF TABLES

Table No.	Description	Page No.
1.1	Comparison of Existing and Proposed Approaches	16
1.2	Summary of Literature Survey	19
1.3	Hardware Requirements	21
1.4	Software Requirements	21
1.5	Comparison with Alternative Recommendation Approaches	22
2.1	Database Tables	29
2.2	REST API Endpoints	30
3.1	Exemplar Movies per Mood	32
3.2	Module Interaction Summary	34
3.3	Test Cases and Results	35
3.4	Deployment and Installation Procedure	35
4.1	Embedding Model Properties	37
4.2	Tools Used in the Project	39
4.3	Communication Layers and Measured Latency	39
4.4	Resource Estimate	40
4.5	Comparison with Alternative Representations	40
5.1	System Configuration Parameters	42
6.1	Recommendation Endpoint Parameters (liked movies are always excluded)	50
7.1	Recommendation Quality at k = 10 (531 users; 95% bootstrap CI for HR)	52
7.2	Paired Difference, Proposed (λ = 0.3, proxy mood) minus B2 (same users)	52
7.3	Effect of λ on HR@10 for Three Mood Sources (oracle uses test items; upper bound only)	53
7.4	Diagnostics (not proposed methods): Quality Prior and Diversity Filter on the Semantic Centroid	53
7.5	Latency and Resource Measurements	53
7.6	Hybrid Evaluation on Held-Out Users (split 0: 106 test users; 'Coverage' = distinct titles per recommendation slot; last column: mean ± sd of HR@10 over 5 splits)	56
7.7	Paired Differences on the Same Test Users (95% bootstrap CI; users better / worse / equal on HR)	57

Table No.	Description	Page No.
7.8	Popularity Discount γ and Blend Weights Chosen on Validation Users for Each Split (with the variety requirement)	57
7.9	Effect of the Mood Re-rank Strength λ (split 0; oracle uses test items and is an upper bound only)	57
7.10	Accuracy versus Variety: Test Results When the Validation Variety Requirement Is Varied (mean over 5 splits)	58
7.11	Catalogue Connectivity	60

LIST OF ABBREVIATIONS

Abbreviation	Expansion
NLP	Natural Language Processing
SBERT	Sentence-BERT
MiniLM	Mini Language Model (distilled Transformer)
TF-IDF	Term Frequency – Inverse Document Frequency
CF	Collaborative Filtering
HR	Hit Rate
NDCG	Normalised Discounted Cumulative Gain
TMDB	The Movie Database
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
ORM	Object-Relational Mapping
SQL	Structured Query Language
UI	User Interface
CPU	Central Processing Unit
GPU	Graphics Processing Unit
ANN	Approximate Nearest Neighbour
UML	Unified Modeling Language
CSV	Comma-Separated Values
ML	Machine Learning
IDE	Integrated Development Environment
TC	Test Case
λ	Lambda – mood-shift weight

CHAPTER 1
INTRODUCTION

1.1 Overview

Choosing what to watch has become a problem of abundance rather than scarcity. Streaming catalogues contain tens of thousands of titles, and a viewer who opens an application on a tired weeknight is rarely looking for the same experience they wanted on Saturday afternoon. Recommender systems exist to narrow this space, but most of them model a viewer with a single, fixed profile: a list of genres liked in the past, or a row in a ratings matrix. Neither representation captures the fact that the same person wants different things on different days.

This project presents **MovieMind**, a web-based movie recommendation system whose central idea is that **movies, users and moods can all be placed in the same mathematical space**. Every movie is converted into a 384-dimensional semantic vector (an embedding) by a pre-trained Sentence Transformer; a user is represented by the centroid of the embeddings of the movies they have liked; and each mood (happy, sad, excited, relaxed, and so on) is represented by a direction vector in that same space. A recommendation is produced by taking the user's position, nudging it a controllable distance in the direction of the current mood, and returning the nearest movies.

The original idea (v2.0) was summarised by a single equation, which the rest of this report develops and then refines:

$$\textbf{\textit{search_position}} = \textbf{\textit{user_taste}} + \lambda \times \textbf{\textit{mood_direction}}$$

Here λ (lambda) is a number between 0 and 1 that the user can change with a slider. With λ = 0 the system recommends purely from long-term taste; with λ = 1 it recommends purely from the selected mood; values in between let the viewer say "my usual taste, but a little more exciting today". Because the user, the movies and the mood live in the same space, the system can also explain every recommendation geometrically, for example by naming the liked movie that a suggestion is closest to and reporting how much of its ranking came from the mood shift.

The design evolved during the project and the report records both stages. The first version (v2.0) used the idea above literally, moving the search point toward the mood. Evaluation showed that this cost accuracy and that content embeddings alone ignore how well liked a film is, so the final version (v2.1) is a **hybrid**: it blends content similarity with a collaborative signal (what people with similar likes enjoyed) and a quality prior, retrieves a pool of strong

candidates, and then lets the mood and the λ slider **re-rank that pool**. Chapters 3 and 7 describe the final method and give the measurements for both versions.

MovieMind is implemented as a Flask application with a SQLite database, NumPy and scikit-learn for vector arithmetic, and the `sentence-transformers` library for encoding. It runs entirely on a local machine, requires no GPU at query time, and uses no paid API. An earlier version of the system (referred to as v1 in this report) used TF-IDF vectors, genre-based taste profiles and a hard-coded mood-to-genre table; v1 is retained as the baseline against which the proposed approach is evaluated.

1.2 Scope of the Project

MovieMind is designed as a complete, small-scale recommendation product rather than an isolated algorithm. Its scope covers the following areas:

- **Semantic content representation:** converting plot summaries, taglines, genres, keywords, director and cast into dense embeddings for a catalogue of roughly five thousand movies.
- **Personalisation:** maintaining a per-user taste vector that updates whenever a movie is liked or unliked.
- **Mood-aware discovery:** ten selectable moods, each backed by a learned direction vector, with a user-controlled intensity (λ).
- **Explainable output:** every recommendation is returned together with geometry-based reasons (taste match, mood effect, nearest liked movie).
- **Web application:** user registration and login, movie browsing and search, favourites, a profile page and a dashboard containing the mood selector and λ slider.
- **Evaluation:** an offline experiment on the MovieLens dataset comparing the proposed method against three baselines and sweeping λ from 0.0 to 1.0.

The project deliberately does not attempt large-scale deployment, real-time streaming-platform integration or an online user study; these are discussed as future work in Chapter 8.

1.3 Objectives

- To replace sparse TF-IDF movie vectors with dense semantic embeddings so that related concepts (for example "heist", "robbery" and "caper") are recognised as similar even when they share no words.

- To represent each user as a point in the same embedding space as the movies, computed as the centroid of the movies the user likes.
- To learn a direction vector for each of ten moods from the data itself, instead of using a hand-written genre lookup table.
- To rank movies by blending content similarity, a collaborative signal and a quality prior, to let a chosen mood re-rank the strongest candidates with an adjustable strength λ, and to expose λ as a user-facing slider.
- To produce explanations computed from actual vector relationships rather than from fixed text templates.
- To evaluate the approach against baselines using Hit Rate@10, NDCG@10, Diversity@10 and Serendipity@10, and to measure how quality changes as λ varies.
- To keep the whole system lightweight: CPU-only, locally runnable and free of paid services.

1.4 Existing System

The existing system (v1) is the first version of MovieMind. It is a conventional hybrid content-based recommender with the following behaviour:

- **TF-IDF content vectors:** each movie's plot, genres, cast and keywords are converted to a sparse TF-IDF vector; similarity between two movies is the cosine of their vectors.
- **Genre-based taste profile:** a user is represented by a dictionary of genre weights, e.g. {Drama: 0.8, Thriller: 0.4}, updated by an exponential moving average with learning rate $\alpha = 0.2$ whenever a movie is liked.
- **Hard-coded mood table:** each mood maps to a fixed set of genres with boost factors (for example "excited" boosts Action 1.5, Adventure 1.4, Thriller 1.3). Movies whose genres match are multiplied by the boost.
- **Weighted scoring:** the final score is a weighted sum of content similarity, genre match and mood boost, followed by a per-genre diversity cap.
- **Template explanations:** the explanation shown to the user is chosen from fixed sentence templates filled with a genre or movie name.

1.5 Existing System Disadvantages

- **Genre labels do not describe experience:** four films tagged "Thriller" may be a witty heist comedy, a slow-burn occult horror, a twisty domestic drama and a gritty procedural. A genre-based profile treats them as interchangeable.
- **Keyword matching misses meaning:** TF-IDF gives zero similarity to "bank heist" versus "robbery" or "mind-bending" versus "cerebral", because it only counts shared words.
- **Mood and taste are disconnected:** the mood table overrides what the system knows about the user. A viewer who only watches slow dramas and selects "Excited" is shown generic action films they are unlikely to enjoy.
- **Mood is binary:** mood is either applied or not; there is no way to ask for a little more excitement.
- **Explanations are not grounded:** template sentences are not computed from the model, so they can be generic or misleading.

Table 1.1 summarises how the proposed approach compares with the existing system on six practical criteria.

Table 1.1: Comparison of Existing and Proposed Approaches

Criterion	TF-IDF + Genre (v1)	Collaborative Filtering	Proposed MovieMind v2
Understands synonyms / meaning	No (word overlap)	Not applicable	Yes (sentence embeddings)
User representation	Genre-weight dictionary	Row of ratings / latent factors	384-d centroid in movie space
Mood handling	Genre lookup table	Usually none	Direction vector in same space
Mood intensity control	None (on / off)	None	Continuous $\lambda \in [0, 1]$
Cold-start for new movies	Works (content)	Fails (no ratings)	Works (content)
Explanation basis	Fixed templates	Opaque factors	Geometric: taste, mood, nearest liked

1.6 Literature Review

A review of prior published work was carried out to position MovieMind relative to existing research on semantic text embeddings, content-based and embedding-based recommendation,

affective (emotion- and mood-aware) recommendation, and evaluation methodology. Only sources whose existence and bibliographic details were confirmed against a publisher, repository or archive page are cited; the full list is in Chapter 9.

Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks

N. Reimers and I. Gurevych, EMNLP-IJCNLP, 2019

The paper fine-tunes BERT in a siamese setting so that whole sentences are mapped to vectors that can be compared directly with cosine similarity. It reports that finding the most similar pair among 10,000 sentences drops from about 65 hours with raw BERT to about 5 seconds. This is the basis for MovieMind's shared embedding space and its cosine nearest-neighbour search [1].

MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers

W. Wang et al., arXiv:2002.10957, 2020

MiniLM compresses large Transformer models by distilling the self-attention behaviour of the final layer into a much smaller student. The `all-MiniLM-L6-v2` encoder used in this project builds on this compact architecture, which is why encoding a catalogue of several thousand movies is practical on a CPU [2].

The MovieLens Datasets: History and Context

F. M. Harper and J. A. Konstan, ACM TiiS, 2015

This paper describes the origin of the MovieLens datasets, how they are released and the limitations that apply when they are used for research. It justifies the use of MovieLens for the offline evaluation in this project and informs the limitations discussed in Chapter 7 [3].

Content-based Recommender Systems: State of the Art and Trends

P. Lops, M. de Gemmis and G. Semeraro, Recommender Systems Handbook, 2011

The chapter surveys how items and user profiles are represented in content-based systems and how profiles are learned from item descriptions. It supports both the v1 TF-IDF baseline and the idea of building a user profile from the content of liked items [4].

Item2Vec: Neural Item Embedding for Collaborative Filtering

O. Barkan and N. Koenigstein, IEEE MLSP, 2016

Item2Vec adapts the word2vec skip-gram model to learn item embeddings from sets of co-occurring items and reports results competitive with SVD-based collaborative filtering. It supports the premise that items can be represented as points in a vector space where distance reflects similarity [5].

Neural Collaborative Filtering

X. He et al., WWW, 2017

The paper replaces the inner product of matrix factorisation with a learned neural interaction function over user and item embeddings. MovieMind differs by using a fixed, content-derived embedding and a simple centroid for the user, trading some accuracy potential for transparency and cold-start robustness [6].

A Survey of Affective Recommender Systems: Modeling Attitudes, Emotions, and Moods for Personalization

T. Hasan and R. Bunescu, arXiv:2508.20289, 2025

This survey classifies affect-aware recommenders into attitude-, emotion-, mood- and hybrid categories, and discusses how affect is extracted and used. It is the main source used to situate MovieMind's mood-aware design and to frame the gap addressed here: affect is usually handled as a separate signal rather than as a direction in the same space as the content. It is an arXiv preprint [7].

Cumulated Gain-Based Evaluation of IR Techniques

K. Järvelin and J. Kekäläinen, ACM TOIS, 2002

The paper introduces cumulated gain, discounted cumulated gain and their normalised form (nDCG), which reward placing relevant items near the top of a ranking. nDCG@10 is one of the main metrics used in the evaluation plan [8].

Diversity, Serendipity, Novelty, and Coverage: A Survey and Empirical Analysis of Beyond-Accuracy Objectives in Recommender Systems

M. Kaminskas and D. Bridge, ACM TiiS, 2016

The authors survey beyond-accuracy objectives and show how optimising one affects the others. This motivates reporting Diversity@10 and Serendipity@10 alongside accuracy, and suggests that increasing λ may trade some accuracy for novelty [9].

Linguistic Regularities in Continuous Space Word Representations

T. Mikolov, W.-t. Yih and G. Zweig, NAACL-HLT, 2013

The paper shows that semantic relations appear as consistent vector offsets in learned word spaces (the well-known king − man + woman ≈ queen example). This is the direct precedent for treating a mood as an offset, mean(mood exemplars) − mean(all movies), and adding it to a taste vector [10].

Efficient Estimation of Word Representations in Vector Space

T. Mikolov et al., arXiv:1301.3781, 2013

The paper introduces the CBOW and skip-gram models for learning word vectors efficiently from large corpora, and underlies the idea that meaning can be encoded geometrically [11].

Scikit-learn: Machine Learning in Python

F. Pedregosa et al., JMLR, 2011

The paper describes the scikit-learn library, from which the TF-IDF vectoriser and cosine-similarity routines used by the baseline system are taken [12].

Table 1.2 consolidates the literature survey and shows where each source is applied in the project.

Table 1.2: Summary of Literature Survey

Ref.	Contribution	Applied To
[1]	Sentence embeddings comparable by cosine	Movie encoding, nearest-neighbour search
[2]	Compact distilled Transformer encoder	all-MiniLM-L6-v2 model choice
[3]	MovieLens dataset history and limits	Evaluation dataset, limitations
[4]	Content-based profiles from item text	v1 baseline, user centroid
[5]	Item embeddings for recommendation	Items as points in a vector space
[6]	Neural user / item embeddings	Contrast with centroid user model
[7]	Survey of affective recommenders	Research gap, mood modelling
[8]	nDCG ranking metric	NDCG@10
[9]	Diversity and serendipity metrics	Diversity@10, Serendipity@10
[10]	Vector offsets encode relations	Mood direction vector
[11]	Word-vector learning	Embedding-space intuition
[12]	scikit-learn library	TF-IDF baseline, cosine similarity

1.7 Proposed System

- MovieMind v2 encodes every movie into a 384-dimensional vector using the `all-MiniLM-L6-v2` Sentence Transformer applied to a rich text built from plot, tagline, genres, keywords, director and cast. The vectors are computed once, offline, and stored as a NumPy matrix.
- A user's taste is the unit centroid of the embeddings of liked movies, stored as 384 numbers in a new `users.taste_embedding` column (the original genre profile is kept so that v1 can run as a baseline). Unliking a movie recomputes the centroid from the remaining likes.

- Ten mood direction vectors are computed once from the data: for each mood, the mean embedding of a set of exemplar movies minus the mean embedding of all movies, normalised to unit length.
- To recommend, the system blends four signals into a base score (content similarity to the user's taste, a collaborative score from what people with similar likes enjoyed, a smoothed average rating and popularity), removes titles the user has already liked or favourited, retrieves the top 60, re-ranks them with λ × mood fit, applies a diversity filter and returns the top k.
- Each result carries a geometric explanation: taste contribution, mood contribution and the nearest liked movie.
- The user controls λ with a slider on the Home page; the default is 0.3. Because the mood only re-ranks titles the engine already rates highly, it changes the order without lowering match quality.

Figure 1.1 illustrates the idea schematically: movies form neighbourhoods by experience rather than by genre label, the user sits in one neighbourhood, and the mood moves the search point a controlled distance toward another region.

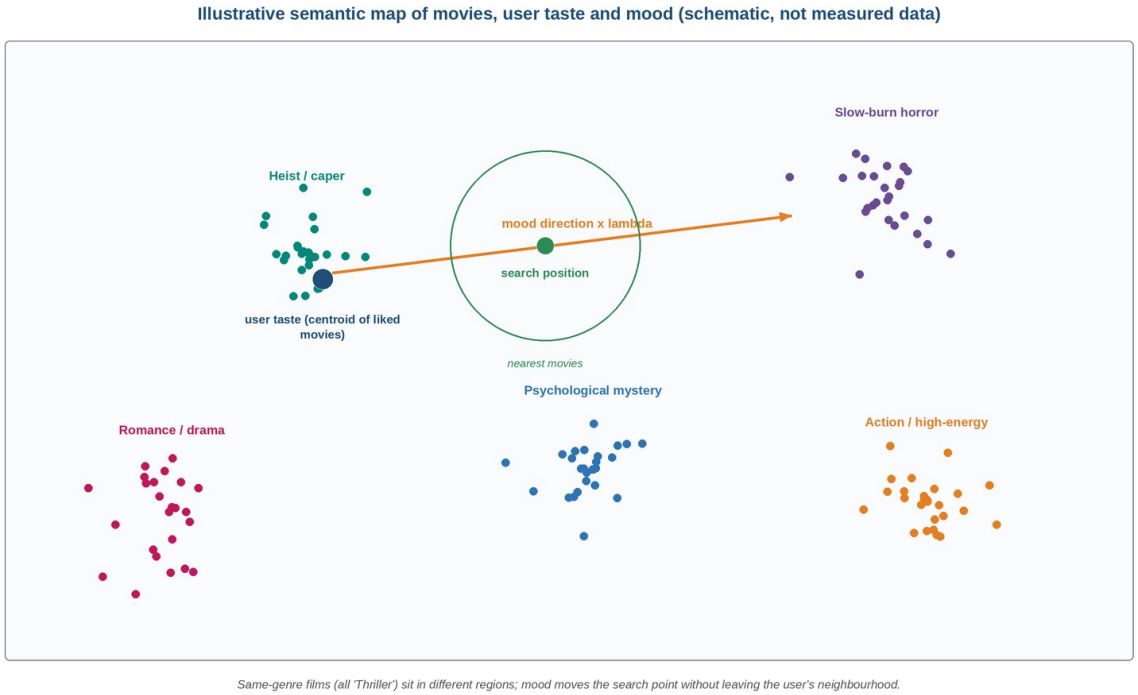


Figure 1.1: Schematic Semantic Map of Movies, User Taste and Mood (illustrative)

1.8 Proposed System Advantages

- **Semantic understanding:** related concepts are close in the space even when they share no words.

- **Mood that respects identity:** the mood shifts the taste vector rather than replacing it, so results stay in the user's neighbourhood.
- **Controllable intensity:** λ gives both the user and the evaluator a continuous control from pure taste to pure mood.
- **Explainable by construction:** explanations are computed from distances in the same space the recommendation was made in.
- **Cold-start friendly:** a new user starts at the global mean embedding; new movies need only a description to be placed in the space.
- **Simple and light:** one recommendation path, one similarity function, one extra dependency (`sentence-transformers`), no GPU at query time and no paid API.

1.9 Hardware and Software Requirements

Tables 1.3 and 1.4 list the minimum hardware and software needed to reproduce the project. No specialised hardware is required.

Table 1.3: Hardware Requirements

Component	Specification	Purpose
Processor	Any modern x86-64 / ARM CPU (4 cores recommended)	Offline encoding, query-time vector maths
Memory	4 GB RAM minimum (8 GB recommended)	Model load (~80 MB) and embedding matrices (~8 MB)
Storage	1 GB free disk	Environment, model cache, SQLite database, datasets
GPU	Not required	Encoding of ~5,000 movies is feasible on CPU
Network	Internet for first setup only	Download dependencies and model; TMDB metadata fetch

Table 1.4: Software Requirements

Software / Library	Version	Purpose
Python	3.10+	Implementation language
Flask, Flask-SQLAlchemy, Flask-Login, Flask-CORS	3.0+	Web framework, ORM, sessions
sentence-transformers	3.0+	all-MiniLM-L6-v2 encoder (new in v2)

Software / Library	Version	Purpose
NumPy, SciPy, scikit-learn	1.26+ / 1.12+ / 1.4+	Vector arithmetic, cosine similarity, TF-IDF baseline
pandas	2.1+	Data preparation and evaluation
SQLite / SQLAlchemy	SQLAlchemy 2.0+	Persistence of movies, users, favourites
Custom CSS + vanilla JavaScript	—	Sky Light design system, mood chips, λ slider, scroll-driven SVG background
pytest, pytest-cov	8.0+ / 4.1+	Automated tests
Git	Any recent	Version control

1.10 Non-Functional Requirements

- **Latency:** a recommendation request should complete in well under one second for a catalogue of about 5,000 movies, because it is a single matrix–vector product. Measured end-to-end through Flask (100 requests, 5 liked movies) it takes a median of 3.6 ms and 95% finish within 5.6 ms (Chapter 7).
- **Reproducibility:** embeddings and mood vectors are generated by a script (`build_embeddings.py`) so that the whole pipeline can be rebuilt from the database.
- **Local operation:** no cloud service or paid API is needed at any stage.
- **Maintainability:** the recommendation logic is isolated in `app/ml/`; routes only call it.
- **Security:** passwords are stored as hashes and all personal endpoints require an authenticated session.
- **Transparency:** every recommendation must be accompanied by a reason that is derived from the model, not invented.

1.11 Comparison with Alternative Approaches

Several alternatives to a shared embedding space were considered during design. Table 1.5 compares them on the factors that mattered for this project.

Table 1.5: Comparison with Alternative Recommendation Approaches

Approach	Needs ratings?	Mood support	Explainability	Suitability
TF-IDF + genres (v1)	No	Genre lookup	Template	Baseline only

Approach	Needs ratings?	Mood support	Explainability	Suitability
Matrix factorisation / CF	Yes (many)	None natively	Low	Weak cold-start; no mood
Neural CF [6]	Yes (many)	Needs extra model	Low	Heavier; needs training data
LLM-based recommender	No	Via prompt	Free text	Needs paid or large model; hard to control
Shared embedding space (chosen)	No (content only)	Direction vector + λ	Geometric	Best fit for a small, local, explainable system

A shared content embedding was chosen because it works with no ratings, treats mood and taste in one geometry, and is cheap to run. Collaborative signals can be added later as a second embedding (Section 8.3).

CHAPTER 2
SYSTEM DESIGN

2.1 General Description

MovieMind is a web application with an offline machine-learning preparation stage and an online recommendation stage. During preparation, movie metadata is turned into embeddings and mood direction vectors. During operation, the user interacts with a browser interface; the Flask back end updates the user's taste vector when movies are liked and, on request, computes hybrid recommendations with mood re-ranking from the stored matrices and statistics.

This chapter describes the design through complementary views: a block diagram of the end-to-end pipeline, a data-flow diagram that separates the offline and online paths, a use-case diagram of the actors, an activity diagram of the recommendation algorithm, and UML collaboration and class diagrams of the software objects.

2.2 Prototype / Block Diagram

Figure 2.1 shows the pipeline from movie metadata to recommendations. Movie metadata is encoded by the Sentence Transformer into an embedding matrix. The recommendation engine combines that matrix with a user taste centroid, collaborative and quality statistics and a mood direction vector into a ranked list, and the Flask API returns the top movies with explanations.

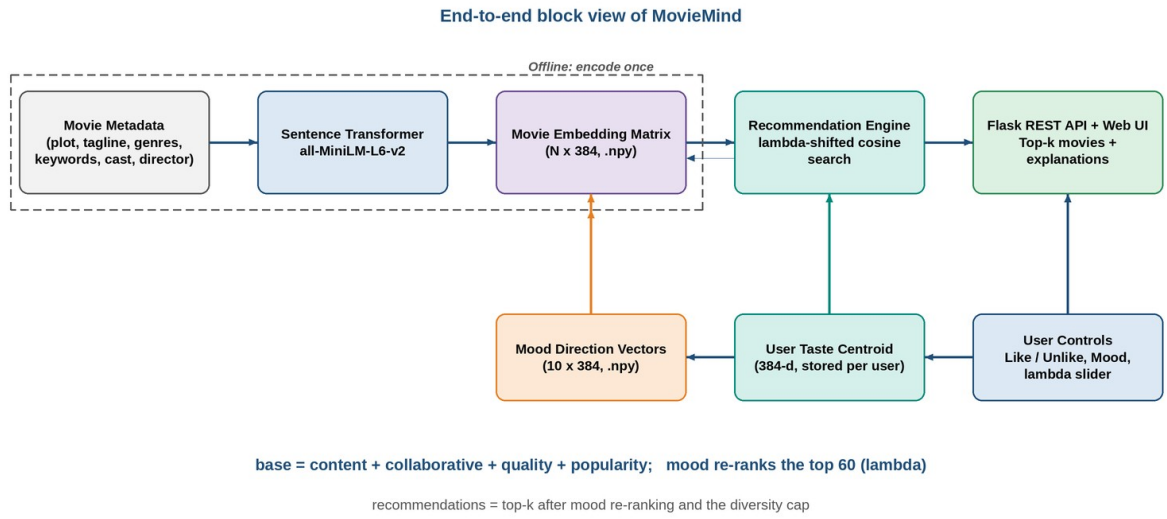


Figure 2.1: System Block Diagram

2.3 System Architecture (Data Flow Diagram)

Figure 2.2 expands the block diagram into a data-flow view. The offline pipeline reads the movie database, builds a rich text per movie, encodes it and stores two matrices:

`movie_embeddings.npy` (N × 384) and `mood_vectors.npy` (10 × 384). The online pipeline updates the taste centroid when a user likes a movie, and on a recommendation request loads the stored matrices and statistics, scores all movies on four components, blends them, retrieves the top 60, re-ranks them by the mood, applies the diversity filter and attaches explanations.

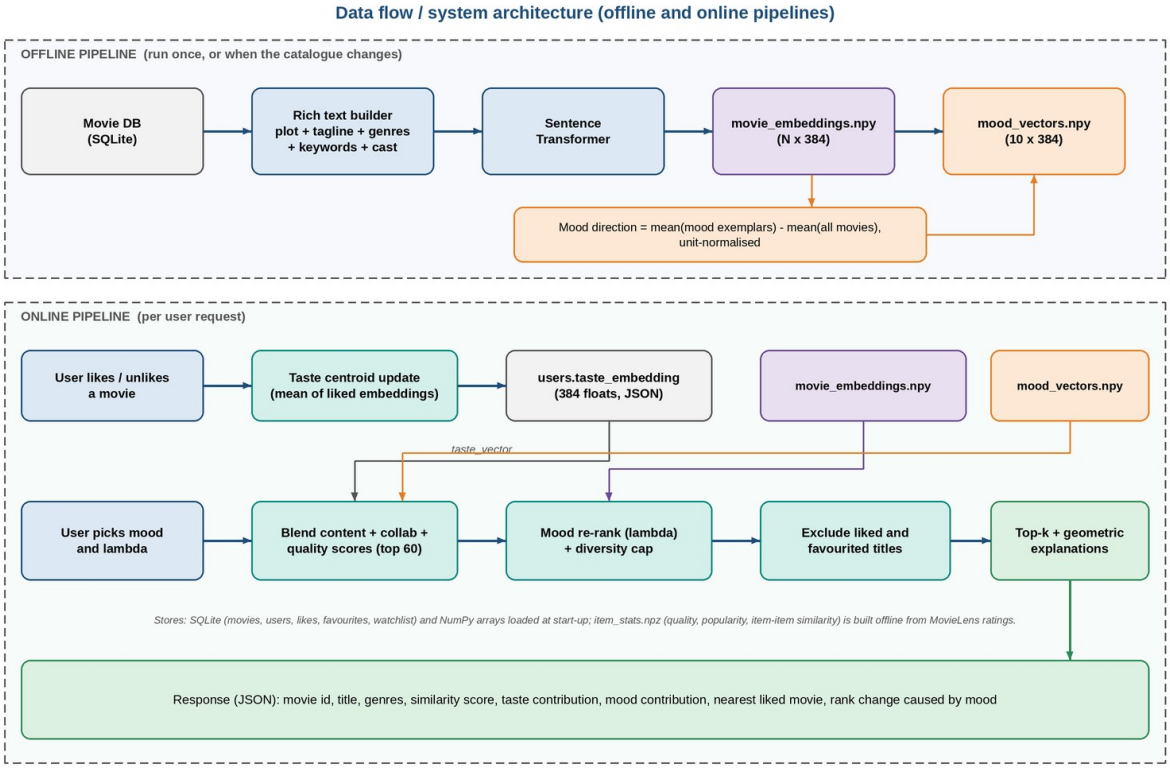


Figure 2.2: Data Flow / System Architecture Diagram

The offline stage runs once per catalogue change. The online stage performs no model inference: it is vector arithmetic over matrices already held in memory, which is why latency stays low.

2.4 Use Case Diagram

Figure 2.3 identifies two actors. The **User** signs up and logs in, browses and searches movies, likes or unlikes movies, selects a mood, adjusts the λ slider, views recommendations and views the explanation for each. Selecting a mood and adjusting λ extend the recommendation view, and viewing an explanation is part of viewing recommendations. The **Administrator** manages the movie catalogue and rebuilds the embeddings and mood vectors when the catalogue changes.

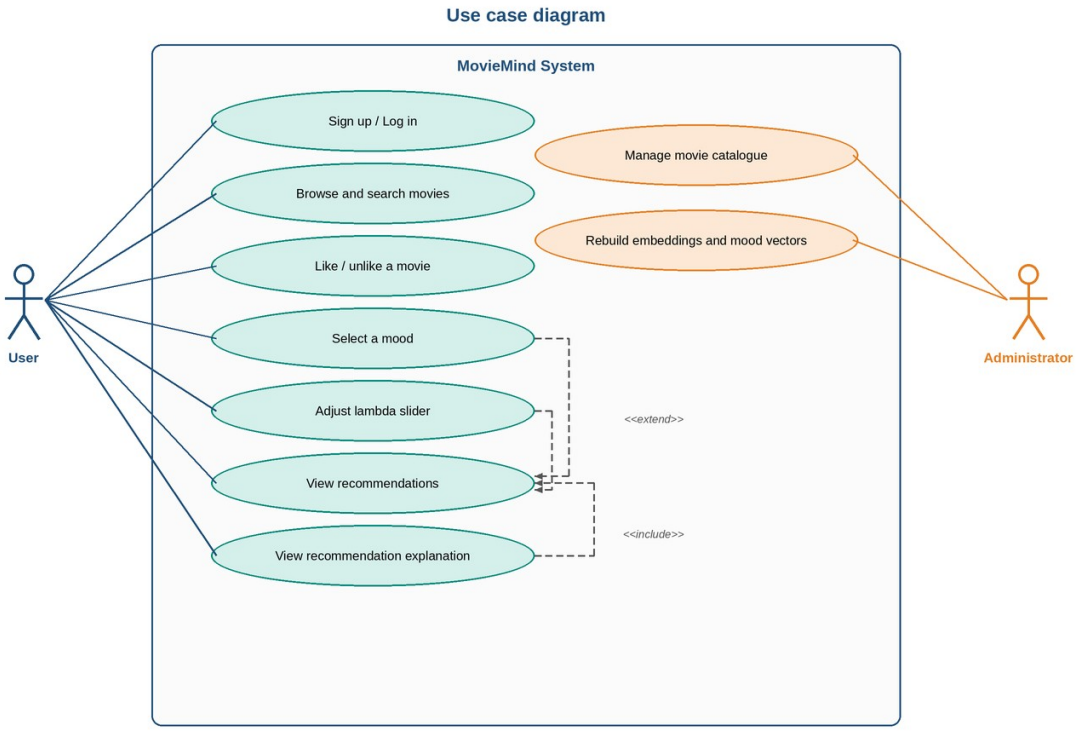


Figure 2.3: Use Case Diagram

2.5 Activity Diagram (Algorithm Flowchart)

Figure 2.4 shows the algorithm executed for each recommendation request. The user's liked titles and taste centroid are loaded (a new user falls back to a centroid of well-rated films in their starting genres and has no collaborative part). Every movie is scored on four components: content similarity to the taste, a collaborative score, a smoothed rating and popularity. The z-scored components are blended with fixed weights into a base score, titles already liked or favourited are removed, and the top 60 candidates are retrieved. If a mood is chosen and λ is greater than zero, the candidates are re-ranked by $(1 - \lambda) \times \text{base} + \lambda \times \text{mood fit}$; otherwise the base order is kept. A diversity cap is applied, the top k are taken and an explanation is computed for each.

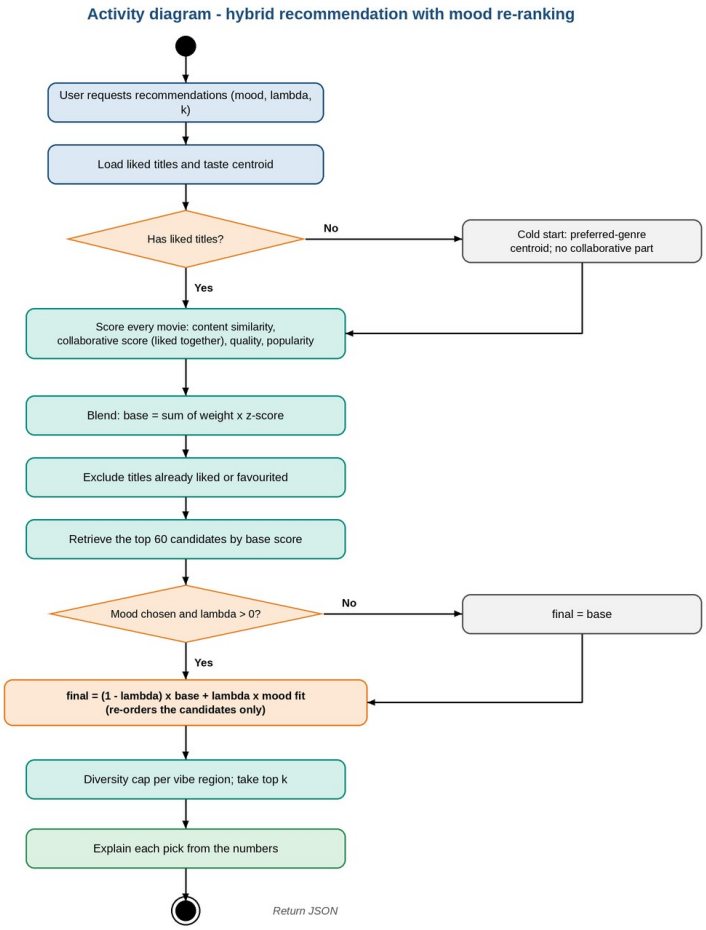


Figure 2.4: Activity Diagram - Hybrid Recommendation with Mood Re-ranking

2.6 UML Collaboration Diagram

Figure 2.5 shows the objects that collaborate on one request. The browser sends the request to the recommendation route (message 1). The route loads the user and taste vector from the database (2) and calls the recommendation engine (3). The engine reads the movie embeddings and mood vector from the embedding store (4) and asks the explainer for contributions (5, 6). The ranked movies with explanations return to the route (7), which sends JSON to the browser (8). Liking or unliking a movie separately triggers the taste updater.

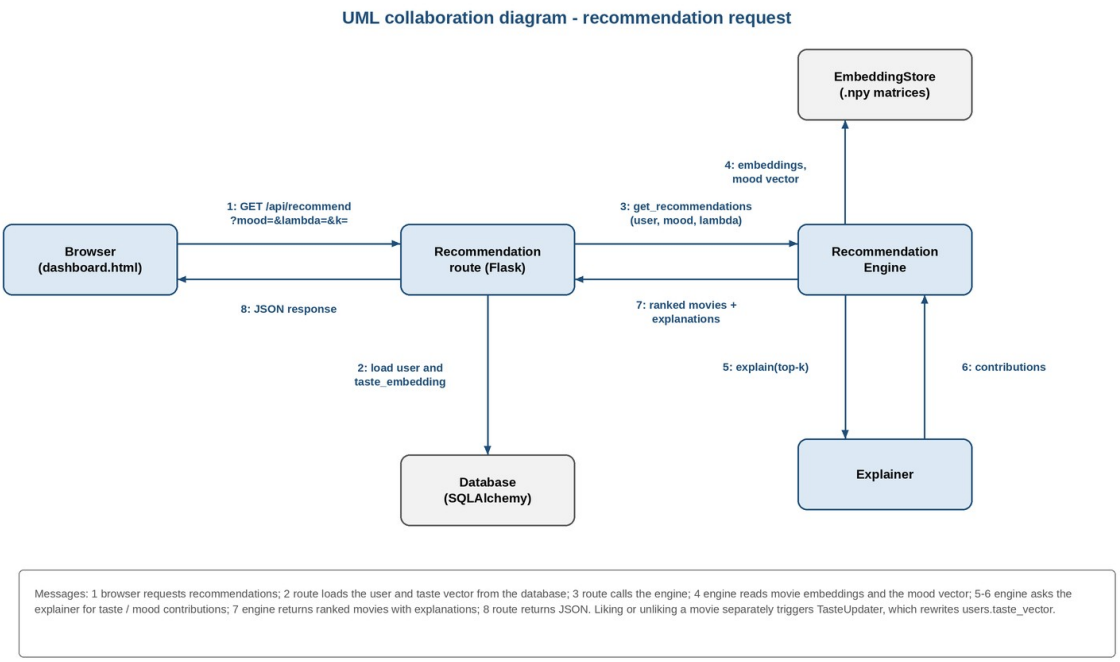


Figure 2.5: UML Collaboration Diagram

2.7 UML Class Diagram

Figure 2.6 shows the key classes. `User`, `Movie`, `UserLike`, `UserFavorite`, `UserWatchlist` and `UserInteraction` are the persistent SQLAlchemy models; `User.taste_embedding` is a new column holding 384 floats as JSON; the v1 genre dictionary in `taste_vector` is kept for the baseline. `EmbeddingStore` loads the two NumPy matrices and exposes embeddings and mood directions. `TasteUpdater` maintains the centroid. `EmbeddingRecommender` performs the hybrid ranking, the mood re-rank and the diversity filter, and `hybrid_explanation` computes the explanation. `build_embeddings.py` is the offline script.

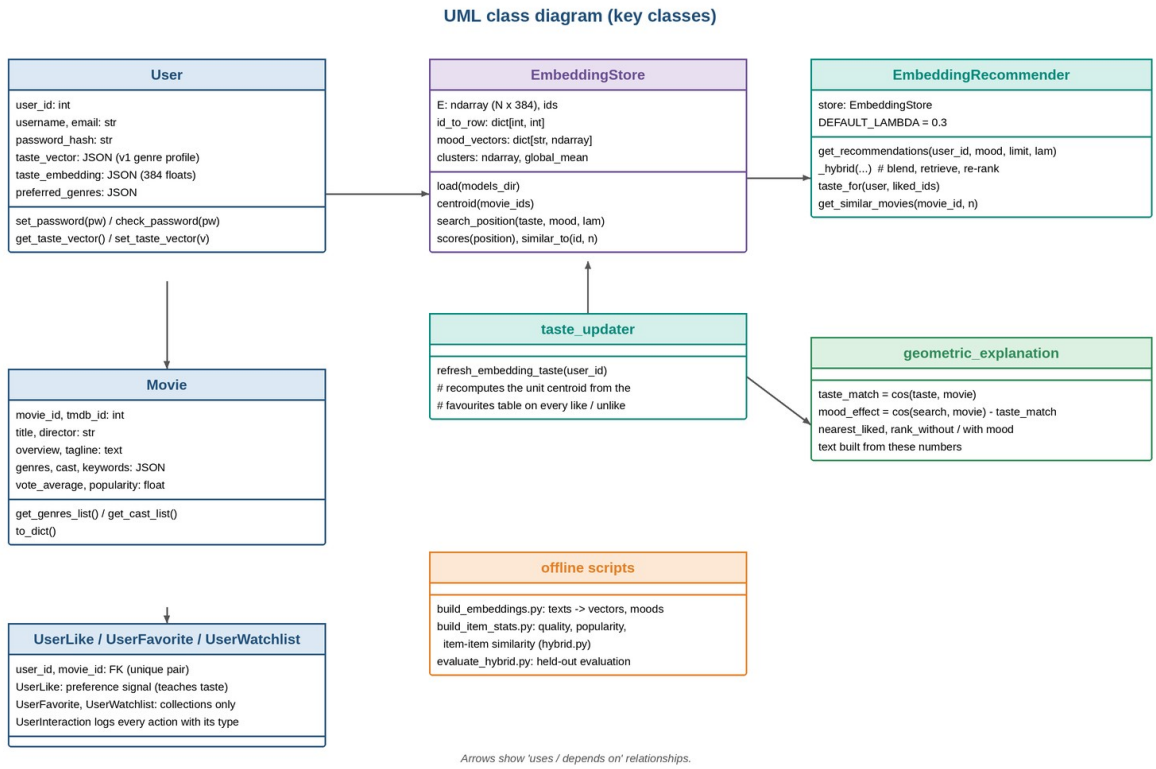


Figure 2.6: UML Class Diagram

2.8 Database Design

The relational schema of v1 is retained. One column is added, `users.taste_embedding`, holding a JSON list of 384 floating-point numbers; a small start-up check adds it to databases created before v2. Table 2.1 lists the tables. Three user actions are kept deliberately separate: a **Like** is the preference signal that teaches the recommender; a **Favorite** adds the title to a curated collection; a **Watchlist** entry saves it for later. Only likes change the taste profile. Databases created before this change are upgraded at start-up, and favourites that the earlier version recorded as likes are restored as likes.

Table 2.1: Database Tables

Table	Key columns	Notes
movies	movie_id (PK), tmdb_id, title, release_year, overview, tagline, genres, cast, keywords, director, vote_average, popularity, poster_path	Metadata from TMDB; genres / cast / keywords stored as JSON text
users	user_id (PK), username, email, password_hash, preferred_genres, taste_vector, taste_embedding	taste_vector: v1 genre profile; taste_embedding: JSON list of 384 floats (v2)
user_likes	id (PK), user_id (FK), movie_id (FK), created_at	Preference signal: the basis of the taste profile and centroid

Table	Key columns	Notes
user_favorites	id (PK), user_id (FK), movie_id (FK), added_at	Curated collection; does not change taste
user_watchlist	id (PK), user_id (FK), movie_id (FK), added_at	Saved for later; does not change taste
user_interactions	id (PK), user_id (FK), movie_id (FK), interaction_type, rating, timestamp	like / unlike / favorite / unfavorite / watchlist / unwatchlist events

2.9 API Design

Existing endpoints are retained and the recommendation endpoint gains optional `lambda` and `mood` parameters. Table 2.2 lists the REST endpoints.

Table 2.2: REST API Endpoints

Endpoint	Method	Description
/api/auth/signup, /login, /logout, /check	POST / GET	Registration, authentication, session check
/api/recommend? mood=&lambda=&limit=	GET	Personalised recommendations with geometric explanations (λ new in v2; 400 if λ is outside 0–1 or the mood is unknown)
/api/similar/<id>	GET	Movies nearest to a given movie in embedding space
/api/movies, /api/movies/<id>	GET	Browse, search, details
/api/moods, /api/genres	GET	Mood options and genre list
/api/user/likes	GET / POST / DELETE	List, add and remove likes (update taste)
/api/user/favorites	GET / POST / DELETE	Curated favourites collection
/api/user/watchlist	GET / POST / DELETE	Titles saved for later
/api/user/state	GET	Ids of liked, favourited and watchlisted titles
/api/user/taste	GET	Current taste vector summary
/api/user/taste/reset	POST	Reset taste vector
/api/user/profile, /history	GET / PUT	Profile and interaction history

CHAPTER 3
SYSTEM IMPLEMENTATION

3.1 Module Description

The system is organised into seven cooperating modules. This decomposition keeps the code maintainable, allows each module to be tested independently and makes it easy to change one part, for example swapping the embedding model, without touching the rest.

3.1.1 Data Preparation Module

This module loads movie records from the SQLite database (populated from TMDB metadata and the MovieLens identifiers) and builds one text string per movie by concatenating the overview, tagline, genres (repeated to give them weight), keywords, director and top cast. Movies without any description are skipped because they would be placed at an arbitrary position in the space. The catalogue used in this project contains 4,988 enriched movies.

3.1.2 Embedding Generation Module

Implemented by `scripts/build_embeddings.py`, this module loads `all-MiniLM-L6-v2`, encodes every movie text in batches and L2-normalises the output so that cosine similarity reduces to a dot product. The result is saved as `movie_embeddings.npy` together with a mapping from movie id to row index. Encoding is a one-time offline step.

3.1.3 Taste Module

This module replaces the genre-based exponential moving average. When a user likes a movie, the taste vector is recomputed as the mean of the embeddings of all liked movies and saved to `users.taste_embedding`. When a movie is unliked, the centroid is recomputed from scratch from the remaining likes rather than by subtracting a vector, which avoids accumulated error. For a user with no likes the global mean embedding is used (cold start). Before the mood shift, the centroid is normalised to unit length so that λ has the same meaning for every user.

3.1.4 Mood Direction Module

For each of the ten moods the module selects exemplar movies using keyword and metadata rules (for example, for "excited": fast-paced, action-packed, high-stakes, adrenaline, relentless) and computes the direction as mean(exemplar embeddings) − mean(all embeddings), normalised to unit length. The ten vectors are stored in `mood_vectors.npy`. Because the directions are learned from the catalogue, they can be recomputed whenever the catalogue changes.

Table 3.1 gives the number of exemplar movies found for each mood in the 4,988-movie catalogue; every mood has several hundred, and the script refuses to build a mood with fewer than 30.

Table 3.1: Exemplar Movies per Mood

Mood	Exemplars	Mood	Exemplars
Happy	617	Thrilled	682
Sad	403	Nostalgic	500
Excited	535	Adventurous	345
Relaxed	481	Thoughtful	501
Romantic	420	Scared	456

Figure 3.1 shows how a mood direction is obtained from the global mean and the mean of the mood's exemplars.

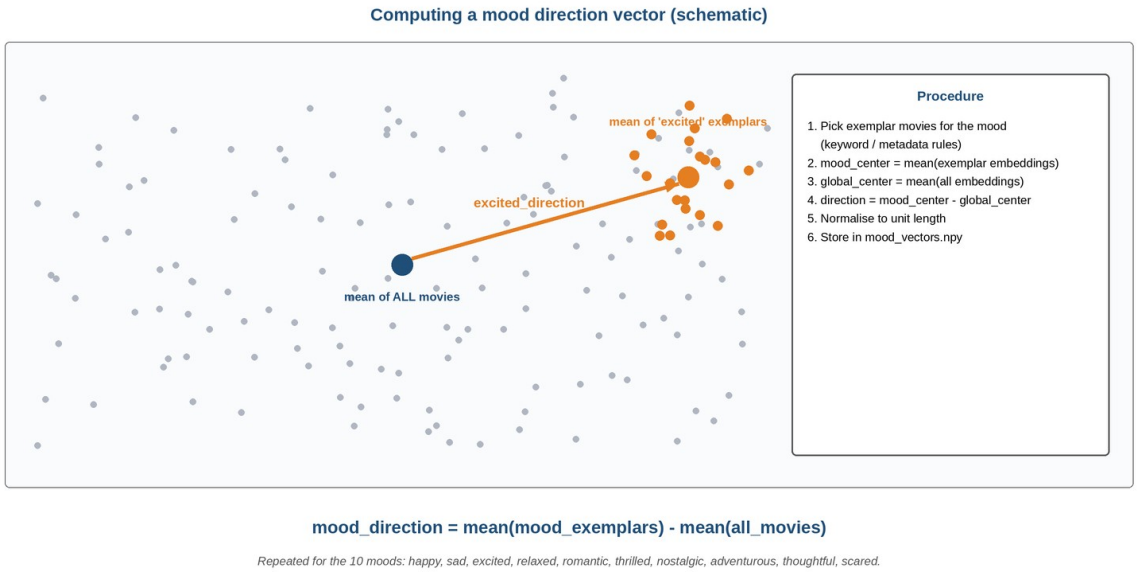


Figure 3.1: Computing a Mood Direction Vector (schematic)

3.1.5 Recommendation Engine Module

Given a user, an optional mood, λ and k, the engine builds four z-scored component vectors over the catalogue (content, collaborative, quality, popularity), combines them with fixed blend weights into a base score, removes titles the user has already liked or favourited, and keeps the 60 highest-scoring candidates. The mood then acts as a **re-ranker**: each candidate's final score is $(1 - \lambda) \times$ min-max(base) $+ \lambda \times$ min-max(cosine(movie, mood direction)). With $\lambda = 0$ the order is exactly the base order, and a mood can never promote a title outside the 60 retrieved, so it cannot trade match quality for mood without limit. The diversity filter then limits how many

results may come from the same region of the space (at most max(2, k/3) per cluster, where clusters are obtained offline by k-means over the movie embeddings). Complexity is O(N × 384) per request.

Figure 3.2 summarises the final ranking pipeline.

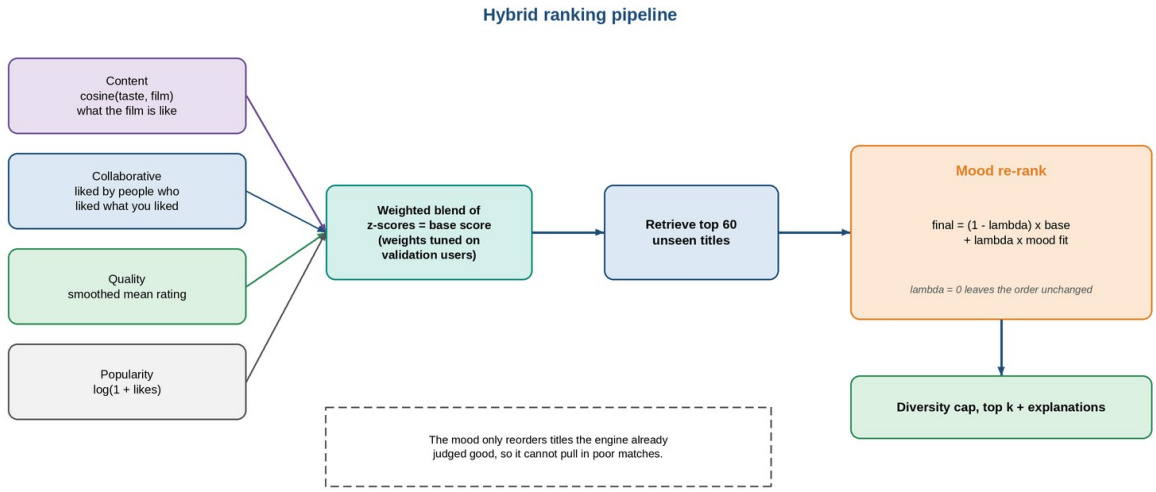


Figure 3.2: Hybrid Ranking Pipeline

3.1.6 Explanation Module

For each recommended movie the module takes the weight × z-score contribution of each of the four components; the largest one is the **reason**. A collaborative reason is shown as "Liked by people who also liked X", a content reason as "Close to X, which you liked" (X being the nearest liked film by cosine similarity), a quality reason as "Highly rated (7.8/10)" and a popularity reason as "Popular with viewers who share your taste". If a mood is active and the re-rank moved the title up, the sentence adds "scared mood lifted it 7 places". The numbers behind each sentence are returned with it, so every explanation can be checked.

3.1.7 Collaborative and Prior Statistics Module

Implemented by `app/ml/hybrid.py` and `scripts/build_item_stats.py`, this module turns the MovieLens ratings into three arrays held in `item_stats.npz` (about 1.1 MB): a **quality** value per movie (its mean rating, shrunk toward the global mean with a prior strength of 20 so that films with few ratings are not over-trusted), a **popularity** value (log of one plus the number of likes, a like being a rating of 4 or more), and the **item-item similarity**, the 100 nearest neighbours of each movie by co-likes divided by (√(n_i × n_j) + 10), a shrunk cosine over the binary liked matrix. A user's collaborative score for a movie is the sum of its similarities to the movies they liked, divided by (number of likes of the movie + 10)^γ. This **popularity discount** (γ = 0.5 in production) stops the raw co-like sum from favouring whatever

many people liked, which would show everyone the same few titles (Section 7.8). Because every statistic comes from a ratings table that the caller chooses, the evaluation can build them from training users only.

3.2 Module Interaction Summary

Table 3.2 consolidates the modules, their responsibilities and the modules they exchange data with.

Table 3.2: Module Interaction Summary

Module	Primary Responsibility	Interacts With
Data Preparation	Build rich text per movie	Embedding Generation
Embedding Generation	Encode text to 384-d vectors, save matrix	Data Preparation, Mood Direction
Taste	Maintain user centroid	Recommendation Engine, Database
Mood Direction	Compute and store mood vectors	Embedding Generation, Recommendation Engine
Statistics	Quality, popularity, item-item similarity from ratings	Recommendation Engine
Recommendation Engine	Blend, retrieve, mood re-rank, diversity	Taste, Mood Direction, Statistics, Explanation
Explanation	Geometric reasons for each result	Recommendation Engine

3.3 Testing Strategy

Testing follows a bottom-up strategy. Unit tests check each module in isolation (shape and normalisation of embeddings, correctness of the centroid and its recomputation on unlike, the identity $\lambda = 0 \Rightarrow$ the base ranking, exclusion of liked movies, the diversity cap, item statistics and mood re-ranking). Integration tests then call the REST API end to end. The v1 project already contained 55 automated tests (`test_api.py`, `test_models.py`, `test_ml.py`); v2 adds 18 in `tests/test_embeddings.py`, 14 in `tests/test_interactions.py` (Like, Favorite and Watchlist semantics) and 12 in `tests/test_hybrid.py` (item statistics, blending, mood re-ranking, explanations), so the suite now has 99 tests, all of which pass. During this work an existing test-isolation fault was also found and fixed: the fixtures changed the database URL after the application had been created, so the tests ran against the real database and `db.drop_all()` would have erased it; `conftest.py` now forces an in-memory database. Table 3.3 lists the principal v2 test cases with their expected and observed results.

Table 3.3: Test Cases and Results

Test ID	Test Case	Expected Result	Observed
TC-01	Embedding matrix shape and norm	N × 384, every row unit length	Pass (4,988 × 384)
TC-02	Synonym similarity ("bank heist" vs "robbery")	Cosine clearly higher than TF-IDF (which is ~0)	Pass
TC-03	New user, no likes	Valid recommendations (cold start)	Pass
TC-04	One like	Taste vector equals that movie's embedding	Pass
TC-05	Unlike a movie	Centroid equals mean of remaining likes	Pass
TC-06	λ = 0 with a mood selected	Ranking identical to no-mood ranking	Pass (also identical in the full evaluation)
TC-07	Liked movies in results	None returned	Pass
TC-08	Diversity cap	No cluster exceeds max(2, k/3)	Pass
TC-09	Mood shift direction	Mood-aligned movies rise in rank as λ increases	Pass
TC-10	λ outside [0, 1]	HTTP 400 with clear message	Pass (also for unknown mood)
TC-11	Explanation fields	taste, mood contribution and nearest liked present	Pass
TC-12	Request latency (5,000 movies)	Well under 1 s on CPU	Pass: 3.6 ms median (API)

3.4 Deployment and Installation Procedure

Table 3.4 gives the procedure to install and run the system on a new machine.

Table 3.4: Deployment and Installation Procedure

Step	Action	Notes
1	Clone the repository and create a virtual environment	python -m venv venv; activate it
2	Install dependencies: pip install -r requirements.txt	Includes sentence-transformers>=3.0.0
3	Copy .env.example to .env and set SECRET_KEY, TMDB_API_KEY	TMDB key only needed to fetch metadata
4	python scripts/setup_database.py, then scripts/load_movies.py	Creates tables and loads movies
5	python scripts/build_embeddings.py	Downloads model once; writes movie_embeddings.npy and mood_vectors.npy

Step	Action	Notes
6	python run.py	Starts Flask on port 5000
7	Open http://localhost:5000, sign up, like movies	Dashboard shows mood selector and λ slider

CHAPTER 4
TECHNOLOGY STACK

4.1 Software Components

MovieMind is a pure software project; no special hardware is used. Figure 4.1 shows the layered architecture of the technology stack, from the presentation layer down to the data sources.

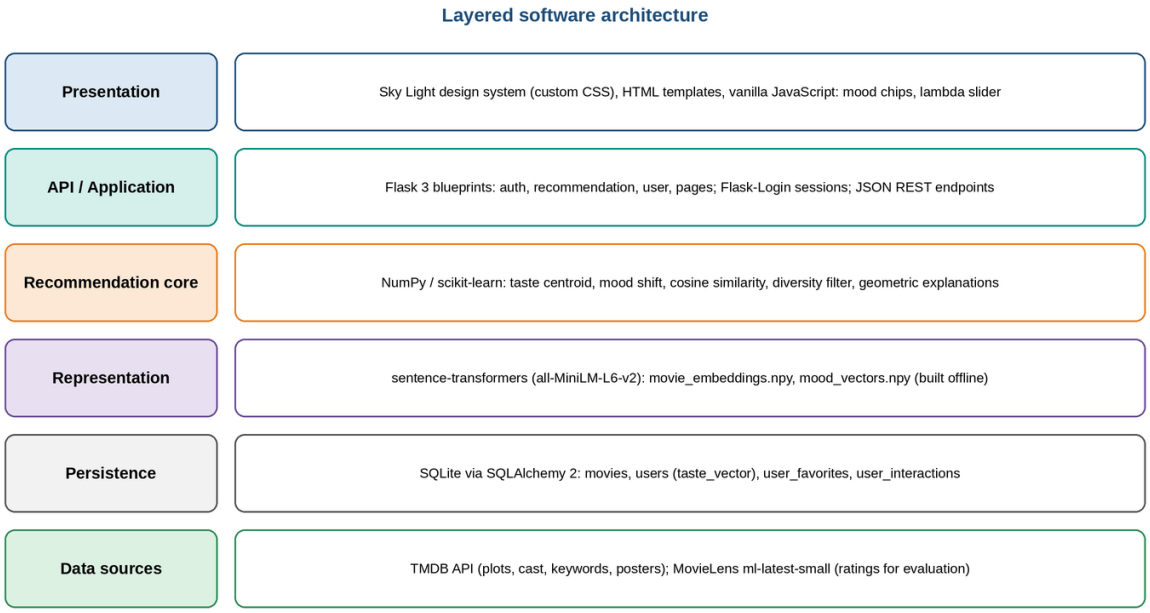


Figure 4.1: Layered Software Architecture

4.1.1 Sentence Transformer (all-MiniLM-L6-v2)

The encoder maps text to a 384-dimensional vector. It is a distilled model [2] with six Transformer layers, trained to produce sentence embeddings suitable for semantic similarity [1]. It was chosen because it is small (about 80 MB), runs on a CPU and gives a good speed-quality trade-off for semantic search. Table 4.1 lists its main properties.

Table 4.1: Embedding Model Properties

Property	Value
Model	sentence-transformers/all-MiniLM-L6-v2
Output dimension	384
Transformer layers	6
Approximate size	~80 MB
Hardware needed	CPU is sufficient
Query-time model use	None (embeddings are pre-computed)
Storage for 4,988 movies	4,988 × 384 × 4 bytes ≈ 7.66 MB

Figure 4.2 shows the encoding pipeline.

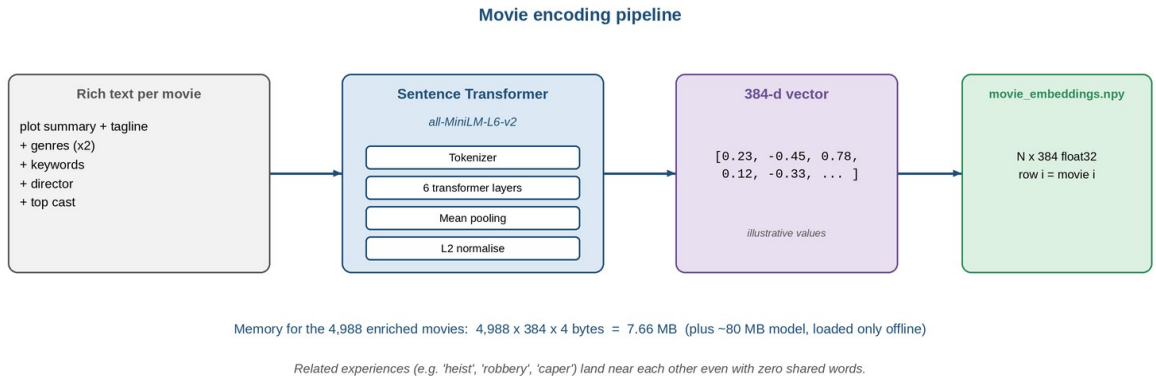


Figure 4.2: Movie Encoding Pipeline

4.1.2 NumPy and scikit-learn

NumPy performs the vector arithmetic: centroids, mean differences, normalisation and the matrix–vector product that yields all cosine similarities. scikit-learn provides the TF-IDF vectoriser and cosine-similarity utilities used by the v1 baseline and the k-means clustering used by the diversity filter [12].

4.1.3 Flask, SQLAlchemy and SQLite

Flask serves the REST API and HTML pages and Flask-Login manages sessions. SQLAlchemy maps Python classes to SQLite tables. SQLite needs no server and suits a single-machine project; the same models can be pointed at another database through the `DATABASE_URL` setting.

4.1.4 Front End

The interface is a purpose-built design system called Sky Light, written in plain CSS and vanilla JavaScript with Inter as the interface typeface. It uses a bright, airy palette defined as CSS custom properties (off-white surfaces, pale sky-blue and ivory secondary surfaces, a muted blue accent, and a rose tone reserved for liked and favourite states), white cards with fine borders, small corner radii and restrained shadows. A compact left sidebar (a bottom tab bar on phones) provides Discover, Movies, Favorites, Watchlist and My Taste. A decorative botanical background of delicate sunlit branches and leaves is drawn as separate transparent SVG layers behind the content; it moves a few pixels while the page scrolls, then eases to rest and stops, so nothing animates while idle, and it is disabled when the user prefers reduced motion. The Discover page offers a mood chip row with a strength slider (λ) and recommendation cards with

a one-line explanation from the engine. Like, Favorite and Watchlist are three distinct actions held in one client-side store, so a title shows the same state on every page. Text meets WCAG contrast, controls are keyboard-operable, and layouts reflow to phone width. Series are not offered because the catalogue contains movies only.

4.1.5 Data Sources

Movie metadata (overview, tagline, cast, keywords, director, poster) comes from the TMDB API [14]. Ratings used for evaluation come from MovieLens [3]. The MovieLens `ml-latest-small` release included in the project contains 100,836 ratings by 610 users on 9,742 movies; 4,988 of these movies have enriched TMDB metadata and form the recommendation catalogue.

4.2 Software Tools

Table 4.2: Tools Used in the Project

Tool	Role in Project
Python 3.10+ / venv	Implementation and environment isolation
sentence-transformers	Loading all-MiniLM-L6-v2 and encoding text
VS Code / Claude Code	Editing and code assistance
pytest	Unit and integration tests
Jupyter	Exploratory analysis and evaluation notebooks
Git	Version control
draw.io / Pillow	Diagrams used in this report

4.3 Communication Protocol Stack

The browser and server communicate over HTTP using JSON for API calls. The back end calls TMDB over HTTPS only during data loading. Inside the server, calls between modules are ordinary Python function calls; the large matrices are loaded once at start-up and reused. Table 4.3 summarises the layers.

Table 4.3: Communication Layers and Measured Latency

Layer	Medium	Protocol / Format	Measured
Browser → Flask	Local network / localhost	HTTP, JSON, session cookie	Not separately measured
Flask → recommender	In-process	Python call	Included below
Recommender compute	In-memory NumPy	Matrix–vector product, sort, diversity	0.48 ms median, 0.57 ms p95

Layer	Medium	Protocol / Format	Measured
Whole /api/recommend request	Flask test client	Auth, SQLite reads, ranking, explanations, JSON	3.6 ms median, 5.6 ms p95
Loader → TMDB	Internet	HTTPS REST (offline step)	Not on request path

4.4 Resource and Cost Estimation

Every component is open source and runs locally, so the monetary cost is zero. Table 4.4 estimates the computing resources.

Table 4.4: Resource Estimate

Item	Estimate	Basis
Embedding matrix	7.662 MB	4,988 × 384 float32 (measured)
Mood vectors	15.36 KB	10 × 384 float32 (measured)
Item statistics	≈ 1.1 MB	item_stats.npz: quality, popularity, 100 neighbours per movie (measured)
Application memory	195 MB peak	Flask + NumPy + scikit-learn with matrices loaded (measured)
Per-user taste vector	≈ 3 KB as JSON	384 floats
Encoding model on disk	≈ 80 MB	Model card
One-time encoding time	137.62 s	CPU only, 4,988 movies (measured)
Per-request compute	O(N × 384) ≈ 1.9 M multiply-adds	N = 4,988
Licence / API cost	None	Open-source components; TMDB free tier for metadata

4.5 Comparison with Alternative Embedding Choices

Table 4.5: Comparison with Alternative Representations

Representation	Dimension	Strength	Drawback	Verdict
TF-IDF	Thousands (sparse)	Fast, simple	No synonyms, no semantics	Baseline (v1)
all-MiniLM-L6-v2 (chosen)	384	Small, CPU-friendly, good quality	English-focused	Best fit
Larger Sentence Transformer	768 and above	Possibly richer	Larger, slower encoding	Possible future upgrade

Representation	Dimension	Strength	Drawback	Verdict
Hosted embedding API	Varies	Strong quality	Paid, needs network, privacy concerns	Rejected

4.6 Privacy, Security and Reliability

Passwords are stored only as salted hashes. All user-specific endpoints require login. The only personal data stored is the username, e-mail and the list of liked movies; the taste vector is a numerical summary of those likes. Because the model and data run locally, no viewing data leaves the machine. For reliability, the matrices are loaded once and validated at start-up (row count must equal the number of encoded movies), and the application falls back to popularity-ranked results if the embeddings are missing.

4.7 Scalability

A full scan costs O(N × 384) per request, which is trivial for thousands of movies and still practical for hundreds of thousands. Beyond that, an approximate nearest-neighbour index (for example FAISS or HNSW) can replace the full scan without changing the hybrid ranking, because only the query vector changes.

4.8 Overall Technical Justification

The stack was chosen to meet four needs: semantic quality (Sentence Transformer), transparency (vector geometry instead of a learned black box), low cost (local, free components) and ease of extension (single recommendation path). Each of these also supports the evaluation, because every design choice can be switched off, for example $\lambda = 0$ for "no mood" or TF-IDF for "no embeddings", to isolate its effect.

CHAPTER 5
SCREENSHOTS

5.1 System Configuration

Table 5.1 records the configuration used for the v2 system, so that the screenshots and results that follow can be interpreted precisely.

Table 5.1: System Configuration Parameters

Parameter	Value
Embedding model	all-MiniLM-L6-v2 (384-d)
Catalogue size	4,988 movies
Moods	10 (happy, sad, excited, relaxed, romantic, thrilled, nostalgic, adventurous, thoughtful, scared)
Default λ	0.3 (range 0.0 to 1.0); re-ranks the top 60 candidates
Collaborative popularity discount γ	0.5
Blend weights	content 0.20, collaborative 0.46, quality 0.03, popularity 0.32
Recommendations per request (k)	10
Diversity cap	max(2, k/3) per cluster
Similarity	Cosine (content); co-like cosine with shrinkage (collaborative)
Backend / DB	Flask 3, SQLite

5.2 Application Screenshots

[The three images below are placeholders for screenshots of the running application; replace them with captures taken from the final v2 build.]



Figure 5.1: Login and Sign-up Page [placeholder]



Figure 5.2: Dashboard with Mood Selector and λ Slider [placeholder]



Figure 5.3: Recommendations with Geometric Explanations [placeholder]

5.3 Console Output

The offline build prints the shapes, timing and exemplar counts. The following is the real output of `python scripts/build_embeddings.py` on the project machine (CPU only):

```
Encoding 4988 movies with all-MiniLM-L6-v2 ...
  embeddings (4988, 384) float32 in 137.6s  (7.66 MB)
  mood exemplar counts: {"happy": 617, "sad": 403, "excited": 535, "relaxed": 481,
    "romantic": 420, "thrilled": 682, "nostalgic": 500, "adventurous": 345,
    "thoughtful": 501, "scared": 456}
  20 clusters; sizes min/max = 133/433
Saved to data/models
```

5.4 System Response and Validation

The screenshots together show the behaviour verified during development:

- the dashboard accepts a mood and a λ value and re-ranks results when either changes;
- each recommendation shows its explanation (taste match, mood effect, nearest liked movie);
- setting λ to 0 produces the same list regardless of the selected mood;
- the offline script reports the matrix shapes and time taken.

CHAPTER 6
SAMPLE CODING

This chapter lists the core Python code of the v2 recommendation pipeline. The listings are extracted automatically from the project source files, so they are exactly the code that produced the results in Chapter 7. The code is organised in the same modules as Chapter 3.

6.1 Embeddings and Mood Vectors

```
# app/ml/embeddings.py (offline functions)

def movie_text(movie):
    """Rich text for one movie: plot + tagline + genres (x2) + keywords + director + top
cast."""
    genres = ' '.join(movie.get_genres_list())
    parts = [movie.overview, movie.tagline, genres, genres,
             ' '.join(movie.get_keywords_list()), movie.director,
             ' '.join(movie.get_cast_list()[:5])]
    return ' '.join(p for p in parts if p)

def encode_texts(texts, model_name=MODEL_NAME, batch_size=64, show_progress=True):
    """Encode texts to L2-normalised float32 embeddings (cosine == dot product)."""
    from sentence_transformers import SentenceTransformer
    model = SentenceTransformer(model_name, device='cpu')
    emb = model.encode(texts, batch_size=batch_size, normalize_embeddings=True,
                       show_progress_bar=show_progress, convert_to_numpy=True)
    return emb.astype('float32')

def mood_exemplar_rows(keyword_lists, mood):
    """Row indices of movies whose TMDB keywords hit this mood's rule set."""
    rules = set(MOOD_KEYWORDS[mood])
    return [i for i, kws in enumerate(keyword_lists) if rules.intersection(k.lower() for k in
kws)]

def compute_mood_vectors(emb, keyword_lists, min_exemplars=MIN_EXEMPLARS):
    """mood_direction = unit( mean(exemplars) - mean(all) ). Returns (matrix, counts)."""
    global_mean = emb.mean(axis=0)
    vecs, counts = [], {}
    for mood in MOODS:
        rows = mood_exemplar_rows(keyword_lists, mood)
        counts[mood] = len(rows)
        if len(rows) < min_exemplars:
            raise ValueError(f"mood '{mood}' has only {len(rows)} exemplars (<
{min_exemplars}); widen its keywords")
        d = emb[rows].mean(axis=0) - global_mean
        vecs.append(d / np.linalg.norm(d))
    return np.vstack(vecs).astype('float32'), counts

def compute_clusters(emb, n_clusters=N_CLUSTERS, seed=0):
    """K-means 'vibe regions' used by the diversity filter."""
    from sklearn.cluster import KMeans
    return KMeans(n_clusters=n_clusters, n_init=10,
random_state=seed).fit_predict(emb).astype('int32')
```

6.2 Embedding Store

```
# app/ml/embeddings.py (EmbeddingStore methods)

def unit(v):
    v = np.asarray(v, dtype='float32')
    n = float(np.linalg.norm(v))
    return v / n if n > 0 else v

centroid(self, movie_ids):
    """Unit centroid of the given movies' embeddings; global mean if none are known."""
    rows = [self.id_to_row[int(m)] for m in movie_ids if int(m) in self.id_to_row]
    return unit(self.E[rows].mean(axis=0)) if rows else self.global_mean.copy()

scores(self, position):
```

```
    """Cosine similarity to every movie (rows are unit length)."""
    return self.E @ unit(position)
```

6.3 Statistics and Hybrid Ranking

```
# app/ml/hybrid.py

def build_item_stats(item_rows, ratings, user_ids, n_items, k=100, shrink=10.0,
prior_strength=20.0):
    """
    Item statistics from a set of ratings (arrays of equal length: item_rows, ratings,
user_ids).

    Returns a dict of arrays:
      quality  Bayesian-smoothed mean rating per item (shrunk toward the global mean)
      pop      log(1 + number of likes) per item
      nbr_idx  (n_items, k) indices of each item's most similar items
      nbr_sim  (n_items, k) their similarity: co-likes / (sqrt(n_i * n_j) + shrink)
    """
    item_rows = np.asarray(item_rows); ratings = np.asarray(ratings, dtype='float64');
user_ids = np.asarray(user_ids)
    mu = ratings.mean() if len(ratings) else 3.5
    cnt = np.bincount(item_rows, minlength=n_items).astype('float64')
    tot = np.bincount(item_rows, weights=ratings, minlength=n_items)
    quality = (tot + prior_strength * mu) / (cnt + prior_strength)

    liked = ratings >= LIKE_THRESHOLD
    _, uidx = np.unique(user_ids[liked], return_inverse=True)
    M = sp.csr_matrix((np.ones(liked.sum(), dtype='float32'), (uidx, item_rows[liked])),
                      shape=(uidx.max() + 1 if liked.any() else 1, n_items))
    M.data[:] = 1.0
    n_like = np.asarray(M.sum(axis=0)).ravel().astype('float64')
    pop = np.log1p(n_like)

    co = (M.T @ M).toarray().astype('float32')
    np.fill_diagonal(co, 0.0)
    denom = np.sqrt(np.outer(n_like, n_like)).astype('float32') + shrink
    sim = co / denom
    np.fill_diagonal(sim, -1.0)                       # an item is never its own neighbour
    k = min(k, n_items - 1)
    idx = np.argpartition(-sim, k, axis=1)[:, :k]
    vals = np.maximum(np.take_along_axis(sim, idx, axis=1), 0.0)
    order = np.argsort(-vals, axis=1)
    return {'quality': quality.astype('float32'), 'pop': pop.astype('float32'),
            'nbr_idx': np.take_along_axis(idx, order, axis=1).astype('int32'),
            'nbr_sim': np.take_along_axis(vals, order, axis=1).astype('float32')}

def cf_scores(stats, liked_rows, n_items):
    """Sum of item-item similarities from the user's liked movies to every movie."""
    s = np.zeros(n_items, dtype='float64')
    if len(liked_rows):
        rows = np.asarray(liked_rows)
        np.add.at(s, stats['nbr_idx'][rows].ravel(), stats['nbr_sim'][rows].ravel())
    return s

def user_components(store, stats, taste_unit, liked_rows, priors=None, cf_gamma=0.0,
cf_offset=10.0):
    """
    The four z-scored component vectors for one user.

    cf_gamma discounts the collaborative score by item popularity: cf / (n_likes + cf_offset)
** cf_gamma.
    A raw co-like sum favours films that many people liked, so everyone is shown the same few
titles;
    gamma = 0 keeps the raw sum, larger values surface less-rated films that are still liked
together.
    """
    priors = priors or prior_components(stats)
    content = zscore(store.E @ np.asarray(taste_unit, dtype='float32'))
    n = store.E.shape[0]
    if len(liked_rows):
        raw = cf_scores(stats, liked_rows, n)
        if cf_gamma:
            raw = raw / ((np.expm1(stats['pop'].astype('float64')) + cf_offset) ** cf_gamma)
        cf = zscore(raw)
    else:
```

```
        cf = np.zeros(n)
    return {'content': content, 'cf': cf, 'quality': priors['quality'], 'pop': priors['pop']}

def combine(comps, weights):
    return sum(weights.get(k, 0.0) * comps[k] for k in COMPONENTS)

def rerank(store, base, seen_rows, mood_dir=None, lam=0.3, k=10, n_candidates=CANDIDATES):
    """
    Retrieve by base score, re-rank the candidates by mood, apply the diversity cap.
    Returns a list of dicts: row, base, final, base_rank (position in the un-moded ranking),
final_rank.
    """
    b = np.array(base, dtype='float64')
    if len(seen_rows):
        b[np.asarray(seen_rows)] = -np.inf
    order = np.argsort(-b)
    cand = order[:max(n_candidates, k)]
    cand = cand[np.isfinite(b[cand])]
    base_c = b[cand]
    if mood_dir is not None and lam > 0:
        mood_c = store.E[cand] @ np.asarray(mood_dir, dtype='float32')
        final = (1 - lam) * minmax(base_c) + lam * minmax(mood_c)
    else:
        final = minmax(base_c)
    seq = cand[np.argsort(-final, kind='stable')]
    final_by_row = dict(zip(cand.tolist(), final.tolist()))
    base_rank = {int(r): i + 1 for i, r in enumerate(cand)}

    cap, counts, picked = max(2, k // 3), {}, []
    for r in seq:
        c = int(store.clusters[r])
        if counts.get(c, 0) < cap:
            picked.append(int(r)); counts[c] = counts.get(c, 0) + 1
        if len(picked) == k:
            break
    if len(picked) < k:                                    # diversity cap left gaps: fill in
order
        for r in seq:
            if int(r) not in picked:
                picked.append(int(r))
            if len(picked) == k:
                break
    return [{'row': r, 'base': float(b[r]), 'final': final_by_row[r], 'base_rank':
base_rank[r], 'final_rank': i + 1}
            for i, r in enumerate(picked)]
```

6.4 User Taste and Recommendation Engine

```
# app/ml/taste_updater.py

def refresh_embedding_taste(user_id, store=None):
    """
    v2 taste: unit centroid of the embeddings of every movie the user currently likes.

    Always recomputed from the likes table, so an unlike never has to "subtract"
    a vector. A user with no likes gets the global mean embedding (cold start).
    Returns the centroid as a NumPy array, or None if embeddings are not built.
    """
    from app.ml.embeddings import get_store
    store = store or get_store()
    user = User.query.get(user_id)
    if store is None or user is None:
        return None
    liked = [l.movie_id for l in UserLike.query.filter_by(user_id=user_id).all()]
    taste = store.centroid(liked)
    user.set_taste_embedding(taste)
    db.session.commit()
    return taste
```

```
# app/ml/embedding_recommender.py (EmbeddingRecommender)

_hybrid(self, user_id, taste, liked_ids, liked_rows, mood, limit, lam, exclude_favorites):
    store, w = self.store, self.config['weights']
```

```
    comps = H.user_components(store, self.stats, taste, liked_rows, self.priors,
cf_gamma=self.config.get('cf_gamma', 0.0))
    base = H.combine(comps, w)
    seen_ids = set(liked_ids) | {f.movie_id for f in
UserFavorite.query.filter_by(user_id=user_id).all()}
    seen_rows = [store.id_to_row[i] for i in seen_ids if i in store.id_to_row] if
exclude_favorites else \
                [store.id_to_row[i] for i in liked_ids if i in store.id_to_row]
    mood_dir = store.mood_vectors.get(mood) if mood else None
    ranked = H.rerank(store, base, seen_rows, mood_dir, lam, limit,
self.config.get('candidates', H.CANDIDATES))

    movies = {m.movie_id: m for m in
Movie.query.filter(Movie.movie_id.in_([int(store.ids[x['row']]) for x in ranked])).all()}
    liked_titles = {m.movie_id: m.title for m in
Movie.query.filter(Movie.movie_id.in_(liked_ids)).all()} if liked_ids else {}
    results = []
    for x in ranked:
        movie = movies.get(int(store.ids[x['row']]))
        if movie is None:
            continue
        ex = hybrid_explanation(store, x['row'], taste, comps, w, liked_rows, liked_titles,
movie, mood, lam,
                                x['base_rank'], x['final_rank'])
        results.append({
            'movie_id': movie.movie_id, 'title': movie.title, 'release_year':
movie.release_year,
            'genres': movie.get_genres_list(), 'overview': movie.overview, 'poster_url':
movie.get_poster_url(),
            'backdrop_url': movie.get_backdrop_url(), 'runtime': movie.runtime,
'vote_average': movie.vote_average,
            'director': movie.director, 'cast': movie.get_cast_list()[:3],
            'score': round(x['final'], 3), 'explanation': ex['text'], 'explanation_detail':
ex,
        })
    return results
```

6.5 Explanations

```
# app/ml/explainer.py

def hybrid_explanation(store, row, taste, comps, weights, liked_rows, titles, movie, mood,
lam,
                       base_rank=None, final_rank=None):
    """
    Explain a hybrid recommendation from the numbers that produced it.

    comps: the user's z-scored component vectors; weights: blend weights. The reason is the
component that
    contributed most (weight x z-score). The mood phrase comes from how far the mood re-rank
moved the title.
    """
    import numpy as np
    from app.ml.embeddings import unit

    contrib = {k: float(weights.get(k, 0.0) * comps[k][row]) for k in ('content', 'cf',
'quality', 'pop')}
    reason = max(contrib, key=contrib.get)
    m = store.E[row]
    taste_match = float(m @ unit(taste))

    nearest = None
    if liked_rows:
        sims = store.E[liked_rows] @ m
        j = int(np.argmax(sims))
        nearest = {'movie_id': int(store.ids[liked_rows[j]]), 'title':
titles.get(int(store.ids[liked_rows[j]])),
                   'similarity': round(float(sims[j]), 3)}
    near_t = nearest['title'] if nearest and nearest['title'] else None

    if not liked_rows:
        text = 'Well rated and popular with viewers' if reason in ('quality', 'pop', 'cf')
else 'Matches your starting genres'
    elif reason == 'cf':
        text = f'Liked by people who also liked {near_t}' if near_t else 'Liked by people with
similar taste'
```

```
    elif reason == 'content':
        text = f'Close to {near_t}, which you liked' if near_t else 'Close to what you like'
    elif reason == 'quality':
        text = f'Highly rated ({movie.vote_average:.1f}/10)' if movie.vote_average else
'Highly rated'
    else:
        text = 'Popular with viewers who share your taste'

    moved = None
    if mood and lam > 0 and base_rank and final_rank:
        moved = int(base_rank - final_rank)
        if moved > 0:
            text += f' · {mood} mood lifted it {moved} place{"s" if moved != 1 else ""}'
    mood_fit = float(m @ store.mood_vectors[mood]) if mood else None
    return {
        'reason': reason,
        'components': {k: round(v, 3) for k, v in contrib.items()},
        'taste_match': round(taste_match, 3),
        'mood_fit': round(mood_fit, 3) if mood_fit is not None else None,
        'nearest_liked': nearest,
        'rank_without_mood': base_rank, 'rank_with_mood': final_rank, 'places_moved_by_mood':
moved,
        'text': text,
    }
```

6.6 REST Endpoint

```
# app/routes/recommendation.py

def get_recommendations():
    """
    Get personalized recommendations for current user

    Query params:
        lambda: Mood strength 0.0-1.0 (default 0.3)
        mood: Optional mood string (happy, sad, excited, relaxed, romantic, thrilled,
nostalgic, adventurous, thoughtful, scared)
        limit: Number of recommendations (default 10, max 50)

    Returns:
        List of recommended movies with explanations
    """
    # Get query parameters
    mood = request.args.get('mood')
    limit = request.args.get('limit', 10, type=int)
    limit = min(max(limit, 1), 50)  # Clamp between 1 and 50

    # lambda: how far the mood moves the search away from the user's taste (0 = pure taste, 1
= pure mood)
    lam_raw = request.args.get('lambda', DEFAULT_LAMBDA)
    try:
        lam = float(lam_raw)
    except (TypeError, ValueError):
        return jsonify({'error': 'lambda must be a number between 0 and 1'}), 400
    if not 0.0 <= lam <= 1.0:
        return jsonify({'error': 'lambda must be between 0 and 1'}), 400

    try:
        recommender = get_recommender()
        if getattr(recommender, 'engine', '').startswith('v2'):
            recommendations = recommender.get_recommendations(
                user_id=current_user.user_id, mood=mood, limit=limit, lam=lam
            )
        else:  # v1 fallback (no embeddings built)
            recommendations = recommender.get_recommendations(
                user_id=current_user.user_id, mood=mood, limit=limit
            )

        return jsonify({
            'recommendations': recommendations,
            'count': len(recommendations),
            'mood': mood,
            'lambda': lam,
            'engine': getattr(recommender, 'engine', 'v1-tfidf')
        }), 200
```

```
    except ValueError as e:
        return jsonify({'error': str(e)}), 400
    except Exception as e:
        return jsonify({'error': f'Failed to get recommendations: {str(e)}'}), 500
```

6.7 API Quick Reference

Table 6.1: Recommendation Endpoint Parameters (liked movies are always excluded)

Parameter	Type	Default	Meaning
mood	string	none	One of the ten moods; omitted means no mood shift; unknown mood returns 400
lambda	float 0.0–1.0	0.3	Strength of the mood shift; outside the range returns 400
limit	integer	10	Number of recommendations (1–50)

CHAPTER 7
RESULT AND DISCUSSION

Two evaluations are reported. The first (Sections 7.1 to 7.4) measures the original v2.0 design, in which the mood moves the search point and the ranking uses content embeddings alone. Its results motivated the redesign. The second (Sections 7.5 to 7.8) measures the final hybrid v2.1 with a stricter, leakage-free protocol in which the blend weights are tuned on separate validation users and results are reported on untouched test users.

7.1 Initial Experimental Setup (v2.0)

The evaluation is an offline experiment on the MovieLens `ml-latest-small` dataset [3], restricted to the 4,988 catalogue movies that have enriched metadata. These carry 75,303 ratings from all 610 users. For each user, ratings are ordered by time: the earliest 80% form the training history and the latest 20% the test set. A test movie is relevant if the user rated it 4.0 or higher, and a training movie counts as liked on the same rule. All movies the user rated in training are excluded from every system's candidate list. Users with fewer than 8 liked training movies or no relevant test movie are skipped, which leaves **531 of 610 users**. Everything is deterministic (fixed random seed) and reproducible with `python scripts/evaluate.py`.

Systems compared (top-10 lists): **POP** the movies most liked in the training data, as a reference; **B1** the v1 engine (TF-IDF and genre hybrid similarity, genre profile updated by exponential moving average, mood as a genre boost); **B2** semantic centroid with no mood ($\lambda = 0$); **B3** semantic centroid with mood applied as a v1-style genre boost; and **Proposed** the mood-shifted search with $\lambda = 0.3$.

MovieLens contains no mood labels, so a proxy is needed. For each user the *proxy mood* is the mood direction closest (by cosine) to the drift of their recent viewing: the mean embedding of their last 5 liked training movies minus the mean embedding of all their liked training movies. It uses no test data. Two controls are reported: a *random mood* for each user, and, as a clearly labelled **diagnostic that cannot be achieved in practice**, an *oracle mood* chosen from the user's held-out relevant movies, which gives an upper bound on what a perfectly known mood could do. Metrics are HR@10, NDCG@10, Diversity@10 (one minus the mean pairwise cosine of the list) and Serendipity@10 (relevant hits whose genres miss the user's three most-watched genres, divided by 10). 95% confidence intervals come from 2,000 bootstrap resamples of users.

7.2 Initial Result

The headline result is negative for the proposed method under the proxy mood. At λ = 0.3 the mood-shifted search reaches HR@10 = 0.073 and NDCG@10 = 0.010, against 0.109 and 0.013 for the same semantic search with no mood (B2). The paired difference over the same users is HR@10 -0.036 (95% CI -0.064 to -0.009; 16 users better, 35 worse, 480 unchanged), so the loss is statistically distinguishable from zero. A random mood performs about as well as the proxy mood (HR@10 0.083), which shows the proxy carries little usable information about what the user goes on to like.

Two further results stand out. First, the simple popularity reference (HR@10 = 0.373) is far above every content-based system, and the v1 engine (B1, 0.156) is above the semantic centroid (B2, 0.109). Second, when the mood is known exactly (oracle diagnostic) the mechanism works: HR@10 rises from 0.109 at λ = 0 to 0.149 at λ = 0.5 and 0.145 at λ = 0.7, with an interior optimum rather than a monotonic trend. The oracle uses the test items, so these values show that a correctly chosen direction helps, not that the system achieves them.

7.3 Initial Performance Metrics

Table 7.1: Recommendation Quality at k = 10 (531 users; 95% bootstrap CI for HR)

Method	HR@10 (95% CI)	NDCG@10	Diversity@10	Serendipity@10
POP: most liked in training (reference)	0.373 (0.333–0.414)	0.097	0.760	0.018
B1: TF-IDF + genre (v1)	0.156 (0.126–0.186)	0.028	0.768	0.005
B2: semantic centroid, no mood (λ = 0)	0.109 (0.085–0.139)	0.013	0.529	0.000
B3: semantic centroid + genre-table mood	0.083 (0.060–0.105)	0.010	0.550	0.000
Proposed: mood-shifted, λ = 0.3, proxy mood	0.073 (0.053–0.098)	0.010	0.530	0.001
Control: λ = 0.3, random mood	0.083 (0.060–0.107)	0.010	0.536	0.001

Table 7.2: Paired Difference, Proposed (λ = 0.3, proxy mood) minus B2 (same users)

Metric	Mean difference	95% CI	Users better / worse / equal
HR	-0.0358	-0.0640 to -0.0094	16 / 35 / 480
NDCG	-0.0032	-0.0063 to -0.0001	26 / 42 / 463
Diversity	+0.0012	-0.0013 to +0.0039	252 / 279 / 0
Serendipity	+0.0002	-0.0006 to +0.0011	3 / 2 / 526

Table 7.3: Effect of λ on HR@10 for Three Mood Sources (oracle uses test items; upper bound only)

λ	Proxy mood	Random mood	Oracle mood	Diversity (proxy)
0.0	0.109	0.109	0.109	0.529
0.1	0.109	0.111	0.117	0.529
0.3	0.073	0.083	0.121	0.530
0.5	0.077	0.073	0.149	0.532
0.7	0.068	0.068	0.145	0.535
1.0	0.066	0.062	0.119	0.544

Table 7.4: Diagnostics (not proposed methods): Quality Prior and Diversity Filter on the Semantic Centroid

Variant	HR@10	NDCG@10
B2 + quality prior, weight 0.0	0.109	0.013
B2 + quality prior, weight 0.05	0.124	0.016
B2 + quality prior, weight 0.15	0.141	0.021
B2 + quality prior, weight 0.3	0.164	0.027
B2 without the diversity filter	0.092	0.012

Table 7.5: Latency and Resource Measurements

Metric	Value	How measured
Recommender compute (median / p95)	0.48 / 0.57 ms	score + sort + diversity, 200 users
Whole /api/recommend request (median / p95)	3.6 / 5.6 ms	Flask test client, 100 requests, 5 likes, includes DB and explanations
One-time encoding of the catalogue	137.62 s	build_embeddings.py, CPU only
Application memory (peak)	195 MB	Flask + NumPy + scikit-learn with matrices loaded

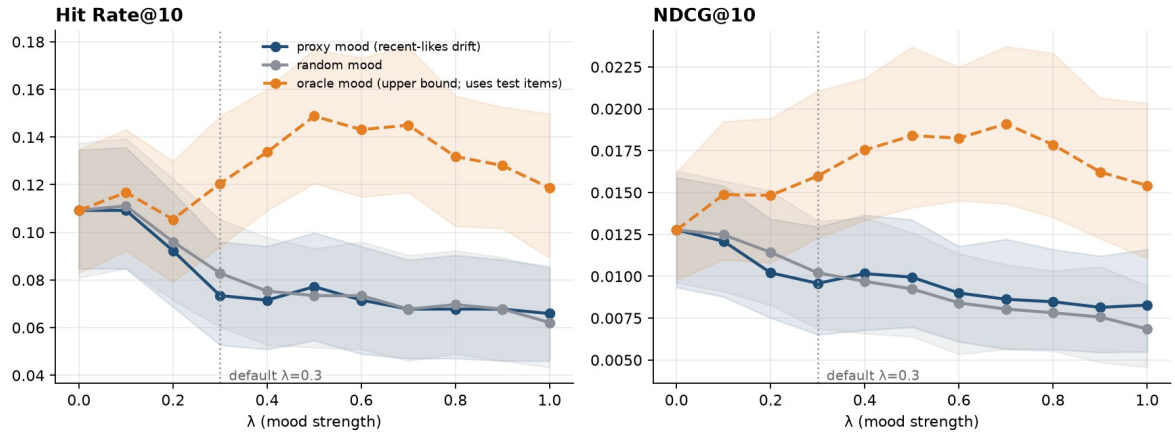


Figure 7.1: HR@10 and NDCG@10 versus λ for proxy, random and oracle moods (shaded: 95% CI)

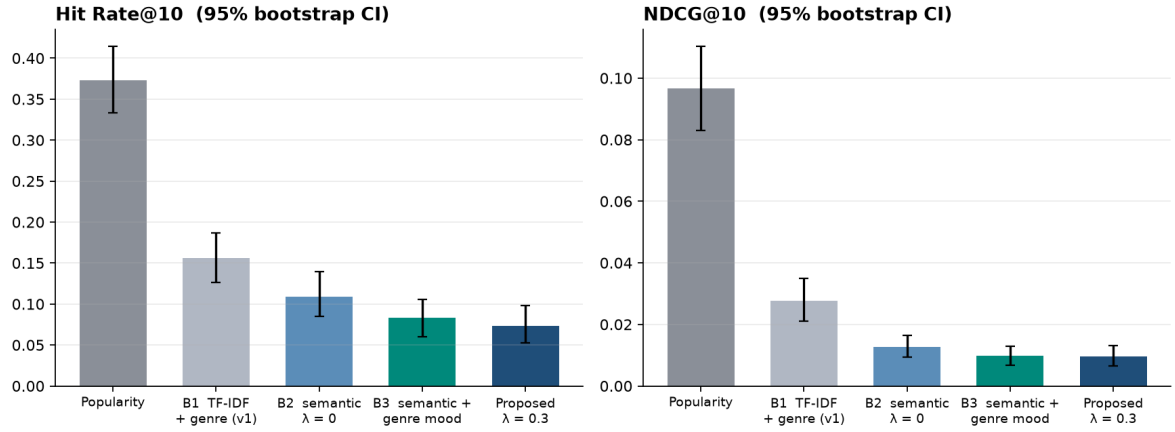


Figure 7.2: Comparison of Baselines and Proposed Method (95% bootstrap CI)

7.4 Discussion of the Initial Evaluation

The four hypotheses stated in advance were checked against the measured results. Two of them are **not supported** and one is supported only in a restricted sense.

- **H1, semantic embeddings beat TF-IDF: not supported on accuracy.** The semantic centroid (B2, HR@10 0.109) is below the v1 engine (B1, 0.156). The diagnostic in Table 7.4 explains why: v1 also scores each movie by its average rating, a popularity-like prior that correlates with whether a user later rates a film 4 or higher, whereas embeddings capture what a film is like, not how well it is regarded. Giving B2 the same kind of prior (weight 0.3, chosen after the fact and not tuned) raises it to 0.164, above B1. Embeddings do match meaning better (Section 1.1, test TC-02), but meaning alone does not predict ratings.

- **H2, a moderate mood shift does not hurt accuracy: not supported with the proxy mood.** HR@10 falls from 0.109 at λ = 0 to 0.066 at λ = 1 and the proxy mood is no better than a random one. Diversity rises only slightly (0.529 to 0.544).
- **H3, mood as a direction beats mood as a genre filter: not demonstrated.** B3 (0.083) and the proposed method (0.073) both sit below B2, and their intervals overlap, so no ordering can be claimed between them.
- **H4, the best λ is interior: supported only for the oracle mood.** With the oracle mood HR@10 peaks at λ ≈ 0.5 to 0.7 (0.149 to 0.145) and falls again toward λ = 1; with the proxy and random moods there is no interior optimum.

What the experiment does and does not show. The machinery behaves as designed: λ = 0 reproduces the no-mood ranking exactly, mood vectors move rankings toward their exemplars, and a correctly chosen mood improves held-out hit rate. What the offline data cannot show is that a mood *the user chooses* improves their experience, because in this dataset the mood is unobserved and the proxy for it is weak. In the application the viewer states the mood and the strength, so the proxy problem is specific to offline evaluation; whether the shifted lists are preferred is a question for a user study, not for ratings accuracy, since a deliberate change of mood is by construction a move away from past taste. The popularity result is a reminder that content-only methods should be combined with a quality or collaborative signal before being judged on accuracy.

7.5 Design Change and Hybrid Evaluation Protocol

Two findings of the initial evaluation shaped the redesign. Content embeddings alone ignore how well liked a film is (Table 7.4: a rating prior lifted the semantic centroid above v1), and moving the search point toward a mood trades accuracy for mood without limit (Table 7.3). The v2.1 hybrid therefore adds a collaborative signal and a quality prior, and turns the mood into a re-ranker over a pool of strong candidates (Section 3.1.5).

The new protocol removes the weaknesses of the first one. **Users are held out:** the 610 MovieLens users are split at random into train (60%), validation (20%) and test (20%) groups, repeated for 5 seeds. The quality, popularity and item-item statistics are built only from the train group, so nothing the evaluated users did can leak into them; this mirrors a new app user meeting knowledge of the crowd. Each evaluated user is scored as before: the earliest 80% of their ratings are their history (liked = rating of 4 or more) and the latest 20% are the targets. The four blend weights are chosen by grid search on the **validation** users (maximising NDCG@10) and the **test** users are used only to report. The first split (seed 0) has 106 validation and 106 test

users and is used for the detailed tables; the other splits show how stable the results are. A time-based split was rejected because only 14 to 23 users have enough history before and activity after any single cutoff.

Accuracy alone rewards popularity, so variety is a requirement. The catalogue audit (Section 7.8) found that the first hybrid, tuned for accuracy only, recommended just 299 distinct films to 600 simulated users. Left free, the weight search always puts a large weight on popularity to win hit rate. The final tuning therefore adds a product requirement fixed in advance: the validation lists must show at least 15% distinct titles per recommendation slot. The collaborative score also gets a popularity discount γ chosen from {0, 0.3, 0.5, 0.7}. For each split, γ and the four weights are the combination with the best validation NDCG@10 that meets the requirement; the test users never take part in the choice. The effect of the requirement is reported as a frontier (Table 7.10).

7.6 Hybrid Results

Table 7.6: Hybrid Evaluation on Held-Out Users (split 0: 106 test users; 'Coverage' = distinct titles per recommendation slot; last column: mean ± sd of HR@10 over 5 splits)

System	HR@10 (95% CI)	NDCG@10	Coverage	HR@10 over 5 splits
Popularity (most liked by the train crowd)	0.387 (0.292–0.481)	0.101	0.061	0.351 ± 0.041
B1: TF-IDF + genre (v1)	0.151 (0.085–0.226)	0.027	0.519	0.153 ± 0.011
Content only (semantic centroid)	0.179 (0.113–0.255)	0.020	0.215	0.135 ± 0.024
Collaborative only	0.462 (0.368–0.557)	0.108	0.147	0.423 ± 0.050
Quality + popularity prior only	0.406 (0.311–0.500)	0.101	0.054	0.355 ± 0.045
Content + prior (no collaborative)	0.236 (0.160–0.311)	0.041	0.192	0.262 ± 0.057
Content + collaborative (no prior)	0.443 (0.349–0.538)	0.105	0.153	0.430 ± 0.037
Hybrid, no mood (final)	0.443 (0.349–0.538)	0.105	0.153	0.439 ± 0.031
Hybrid tuned for accuracy only (no variety requirement)	0.472 (0.377–0.566)	0.119	0.093	0.439 ± 0.039
Hybrid + mood re-rank, λ = 0.3, proxy mood	0.491 (0.396–0.585)	0.109	0.157	0.470 ± 0.033
Control: same, random mood	0.472 (0.377–0.566)	0.108	0.146	0.439 ± 0.041
v2.0 mood-shift (content only), λ = 0.3	0.066 (0.028–0.123)	0.008	0.267	0.078 ± 0.012

Table 7.7: Paired Differences on the Same Test Users (95% bootstrap CI; users better / worse / equal on HR)

Comparison	ΔHR@10	ΔNDCG@10	Users HR better / worse / equal
Hybrid minus popularity	+0.057 (-0.038 to +0.160)	+0.004 (-0.021 to +0.031)	17 / 11 / 78
Hybrid minus v1 (TF-IDF + genre)	+0.292 (+0.179 to +0.406)	+0.079 (+0.047 to +0.113)	39 / 8 / 59
Hybrid minus content only	+0.264 (+0.151 to +0.377)	+0.085 (+0.058 to +0.114)	37 / 9 / 60
Mood re-rank (proxy) minus hybrid without mood	+0.047 (-0.009 to +0.104)	+0.003 (-0.008 to +0.014)	7 / 2 / 97
Mood re-rank (proxy) minus random mood	+0.019 (-0.028 to +0.075)	+0.001 (-0.012 to +0.014)	5 / 3 / 98

Table 7.8: Popularity Discount γ and Blend Weights Chosen on Validation Users for Each Split (with the variety requirement)

Split	γ	Content	Collab.	Quality	Popularity	Val. NDCG@10
Split 0	0.3	0.5	1.0	0.0	0.0	0.106
Split 1	0.5	0.5	1.0	0.0	0.5	0.111
Split 2	0.5	0.0	0.5	0.0	0.5	0.135
Split 3	0.5	0.5	2.0	0.5	1.0	0.139
Split 4	0.5	1.0	1.0	0.0	0.5	0.107
Production (mean of normalised)	0.5	0.195	0.46	0.025	0.32	—

Table 7.9: Effect of the Mood Re-rank Strength λ (split 0; oracle uses test items and is an upper bound only)

λ	HR@10 proxy mood	HR@10 random mood	HR@10 oracle mood	List overlap with no-mood list (proxy)	(oracle)
0.0	0.443	0.443	0.443	1.00	1.00
0.1	0.472	0.472	0.453	0.95	0.95
0.3	0.491	0.472	0.500	0.81	0.83
0.5	0.462	0.453	0.519	0.59	0.61
0.7	0.406	0.425	0.557	0.35	0.35
1.0	0.406	0.434	0.528	0.16	0.16

Table 7.10: Accuracy versus Variety: Test Results When the Validation Variety Requirement Is Varied (mean over 5 splits)

Required coverage	HR@10 (mean ± sd)	NDCG@10	Coverage achieved	Share of slots held by the 100 most-shown titles	γ chosen per split
none (accuracy only)	0.439 ± 0.039	0.116	0.078	1.00	0.0, 0.3, 0.0, 0.0, 0.0
10%	0.471 ± 0.012	0.113	0.116	0.98	0.0, 0.3, 0.5, 0.3, 0.7
15%	0.439 ± 0.031	0.106	0.152	0.93	0.3, 0.5, 0.5, 0.5, 0.5
20%	0.341 ± 0.136	0.077	0.201	0.86	0.7, 0.7, 0.7, 0.7, 0.7
25%	not reachable on this data	—	—	—	—

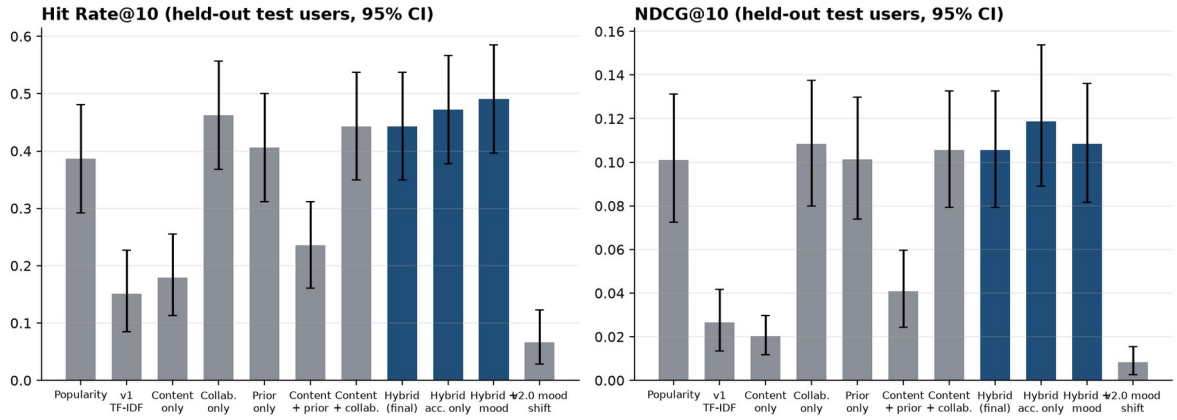


Figure 7.3: Hit Rate@10 and NDCG@10 of the Hybrid Variants on Held-Out Test Users (95% CI)

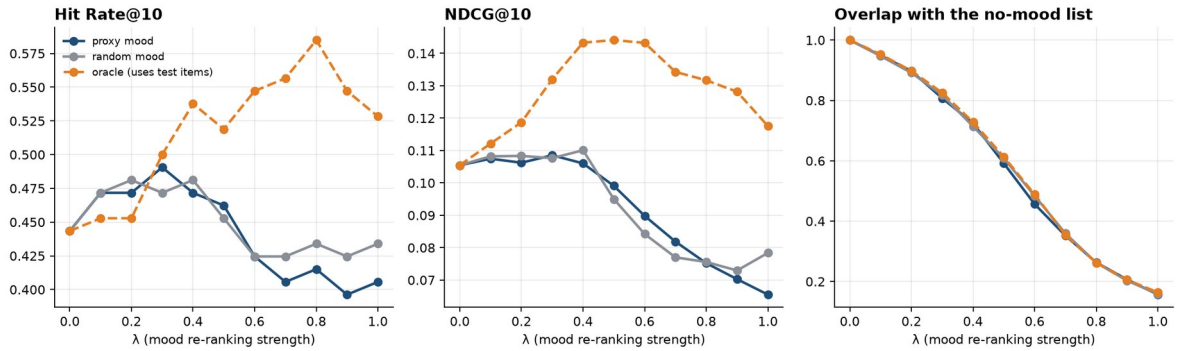


Figure 7.4: Effect of the Mood Re-rank Strength λ on Accuracy and on How Much the List Changes

7.7 Discussion of the Hybrid Results

- **The hybrid is far more accurate than v1 and than content alone.** On held-out users the hybrid reaches HR@10 = 0.443 on split 0 against 0.151 for v1 and 0.179 for content only; the paired gains are +0.292 (95% CI +0.179 to +0.406) and +0.264 (CI +0.151 to +0.377). Over the five splits the means are 0.439 ± 0.031, 0.153 ± 0.011 and 0.135 ± 0.024.

- **Against the popularity reference the evidence is suggestive, not conclusive.** The hybrid is ahead of popularity in all five splits (mean HR difference +0.088, range +0.045 to +0.144), but on split 0 alone the paired interval (-0.038 to +0.160) includes zero, and the NDCG difference is small. It does this while showing 2.8 times as many distinct titles as the popularity list.

- **The collaborative signal does most of the work.** Collaborative only (0.423) is far above content only (0.135); the prior alone (0.355) is about the level of popularity, and content plus prior without collaborative falls to 0.262. Content + collaborative without the prior (0.430) is as good as the full blend (0.439 ± 0.031) within the split-to-split spread. The chosen weights differ between splits (Table 7.8), so production uses their average, which keeps a content share for films that have no ratings. This evaluation cannot show the value of that share, because every catalogue film has ratings.

- **Variety can be bought at little cost in accuracy, up to a point.** Requiring 15% distinct titles per slot leaves mean HR@10 at 0.439 against 0.439 when tuning for accuracy only, while coverage rises from 0.078 to 0.152. A 20% requirement costs accuracy and makes results unstable, and 25% cannot be met on this data (Table 7.10). Variety is therefore limited by the data: half of the catalogue has only one to four ratings.

- **The mood re-rank keeps accuracy while changing the list.** At $\lambda = 0.3$ the top-10 list shares 81% of its titles with the no-mood list, so the mood visibly changes about 2 of ten titles. With the proxy mood HR@10 is 0.491 against 0.443 without mood on split 0 (paired difference +0.047, CI -0.009 to +0.104), and it is slightly ahead in all five splits (mean +0.031; 0.470 against 0.439). A random mood gives 0.439. So the re-rank does not cost accuracy, and the proxy may carry a little real signal, but no single interval excludes zero and this should not be called proven. By contrast the v2.0 design, which moved the search point, scores 0.078 under the same protocol.

- **With an exactly known mood the gain is larger.** With the oracle mood (which uses the test items) HR@10 rises from 0.443 to 0.500 at $\lambda = 0.3$ and 0.557 at $\lambda = 0.7$ (Table

7.9). That is an upper bound; whether a mood chosen by a user improves their experience needs a user study.

In short, the system is now measurably more accurate than the earlier engine and than content-only matching for held-out users, it matches or slightly exceeds the popularity reference while showing a wider range of films, and it explains its picks. It is not shown to be better than popularity by a wide margin, its reach into the long tail of the catalogue is limited by sparse ratings, and satisfaction has not been measured.

7.8 Catalogue Connectivity and Coverage Audit

A separate audit (`scripts/audit_catalogue.py`) checked how well each of the 4,988 catalogue movies is connected to the rest of the system, and whether each can actually be recommended.

Table 7.11: Catalogue Connectivity

Check	Result
Database, embeddings and v1 matrices aligned	4,988 movies in each; identical ids and order: yes
Metadata present (overview / genres / keywords / cast)	100% / 100% / 96.7% / 99.8%
Poster / backdrop path present	100% / 99.2% (40 of 40 sampled poster URLs loaded)
Cluster and mood-fit score for every movie	yes (20 clusters, 10 moods)
MovieLens ratings kept in the catalogue	75,303 of 100,836 (75%); the others belong to films outside the catalogue
Movies with no collaborative neighbour (cold items)	1,424 (28.5%)
Movies with fewer than 5 likes	3,562 (71%)

The plumbing is sound: every movie has a database row, an embedding, a TF-IDF row, a poster path, a cluster and a mood score. The weak point is **reachability**. In a simulation with 600 users each liking 3 to 14 films, the final hybrid recommended 403 distinct films (8.1% of the catalogue) in their top ten, and the 100 most-recommended films filled 87% of all slots; a content-only ranking reaches 592 films. This is a consequence of the data: 71% of films have fewer than five likes, so collaborative evidence exists only for the head of the catalogue. The popularity discount and the variety requirement raised reach from 299 to 403 films, but long-tail discovery remains limited and is a priority for future work.

7.9 Limitations of the Current Prototype

- **Small, old dataset:** MovieLens `ml-latest-small` has 610 users, about 100 test users per split, and ratings from 1996 to 2018. Confidence intervals are wide and the results should be replicated on a larger dataset. The collaborative statistics come from this one crowd, and the live system serves only the 4,988 catalogue films.
- **Offline accuracy is not satisfaction:** hit rate and NDCG measure agreement with later ratings. No user study was run, so whether people prefer the mood-shifted lists is untested.
- **Weak mood proxy:** the drift of recent likes performs like a random mood, so only the oracle diagnostic (which uses test items) shows what a correct mood could add.
- **Limited reach into the catalogue:** collaborative and popularity signals favour well-known films. In simulation the final system recommends about 8% of the catalogue and the 100 most-shown films fill about 87% of slots (Section 7.8); a variety requirement and a popularity discount help but cannot create evidence for films with few ratings.
- **Noisy weight selection:** the blend weights vary between splits (Table 7.8). Production uses their average, which was derived from validation sets that overlap other splits' test users, so the production configuration is not itself a clean test result; the per-split numbers are.
- **Content share untested:** all films have ratings, so the benefit of content similarity for new, unrated films cannot be measured here.
- **Subjective mood exemplars, one mood at a time, English-centred encoder:** as in Chapter 3, mood vectors come from keyword rules, only one mood can be selected, and the encoder is trained mainly on English text.

CHAPTER 8
CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

This project built MovieMind, a movie recommender whose final design (v2.1) is a hybrid: it represents movies, users and moods in one semantic embedding space, blends content similarity with a collaborative signal and a quality prior into a base score, and lets the user's chosen mood re-rank a pool of strong candidates by an adjustable strength λ. Every recommendation is explained from the numbers that produced it. The system was implemented end to end (offline pipeline, taste updates, hybrid ranking, API, a redesigned interface) and verified by 99 automated tests; a request takes a median of 3.6 ms on a CPU.

The evaluation was done twice and both results are reported. The first design, which moved the search point toward the mood using content embeddings alone, trailed a popularity reference (HR@10 0.073 against 0.373) and the earlier TF-IDF engine. After the redesign, and under a stricter protocol in which blend weights are tuned on separate validation users and reported on held-out test users, the hybrid reaches HR@10 of 0.439 ± 0.031 over five splits, against 0.153 for v1, 0.135 for content alone and 0.351 for popularity, and it is ahead of popularity in every split. The mood re-rank changes about 2 of ten titles at the default strength without a measurable loss in accuracy, and with an exactly known mood it improves hit rate.

What is and is not established. Supported: a working, explainable, mood-controllable recommender that is measurably more accurate than the engine it replaced and than content-only matching on held-out users, with the collaborative signal as the main driver. Not established: that the hybrid is substantially better than the best simple alternatives, that a mood chosen by a user improves their experience (the offline mood proxy is no better than chance), and user satisfaction in general. The dataset is small, no user study was run, and weight selection is noisy. These are the first things to address in future work.

8.2 Applications

- **Streaming and video-on-demand platforms:** a mood slider on the home screen that re-ranks a personalised list.
- **Digital libraries and cinemas:** content-based suggestions for catalogues with little rating data.
- **Education and research:** a transparent reference system for studying affect-aware recommendation.

- **Other media domains:** the same hybrid ranking with mood re-ranking can be applied to music, books or podcasts by changing the encoder and mood exemplars.

8.3 Future Enhancement

8.3.0 Stronger Collaborative Models

Replace the item-item co-like similarity with matrix factorisation or a neural collaborative model, and learn the blend weights jointly instead of by grid search. The collaborative signal drives most of the accuracy (Chapter 7), so this is the most promising next step.

8.3.1 Multi-Mood Blending

Allow blends such as 60% excited and 40% nostalgic, so that the mood fit used for re-ranking is a weighted sum of mood directions.

8.3.2 Learning λ per User

Learn the best λ for each person from whether they like mood-shifted recommendations, instead of using one global default.

8.3.3 Multi-Interest and Rating-Aware Taste

Represent a user by two or three taste centres instead of one average, weight each like by its rating and recency, and use dislikes as a negative signal.

8.3.4 Temporal Mood Decay

Let λ fade gradually after a mood is selected, reflecting how a transient emotional state passes.

8.3.5 Multilingual Support

Use a multilingual encoder so that non-English descriptions are represented properly.

8.3.6 Crowd-Sourced Mood Labels

Replace keyword-based exemplars with labels collected from viewers, and recompute directions periodically.

8.3.7 User Study and Real Mood Data

Collect explicit mood choices and satisfaction from real participants, which is the only way to test whether mood-shifted lists are preferred and to replace the weak offline proxy.

8.3.8 Cross-Domain Application

Apply the framework to music, books or podcasts.

8.4 Societal and Ethical Considerations

Privacy. Because encoding and recommendation run locally and the stored taste vector is only a numeric summary of likes, the design is privacy-friendly compared with services that send viewing data to third parties. Passwords are stored as hashes and personal endpoints need authentication.

Filter bubbles. A pure-taste recommender can keep showing similar content. λ and the diversity filter let the user choose how far to move from their usual neighbourhood, which gives the user control rather than hiding the trade-off.

Bias. Embeddings inherit biases of their training text and of the catalogue, which over-represents popular and English-language films. Results should be audited before any real deployment.

Emotional inference. The system uses a mood the user selects; it does not infer emotion from behaviour or sensors, which avoids intrusive affect detection.

Environmental cost. All heavy computation is a one-time CPU encoding of a few thousand short texts; query-time cost is a single matrix product, so the energy footprint is small.

8.5 Deployment Considerations

A production deployment would need an approximate nearest-neighbour index for large catalogues, incremental encoding of newly added movies, monitoring of recommendation quality and latency, and periodic recomputation of mood directions. A phased rollout, beginning with an opt-in mood slider and comparing engagement with the existing list, would show whether the approach helps real viewers before it is widened.

CHAPTER 9
REFERENCES

[1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), Hong Kong, China, 2019, pp. 3982–3992, doi: 10.18653/v1/D19-1410.

[2] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," arXiv:2002.10957, 2020.

[3] F. M. Harper and J. A. Konstan, "The MovieLens datasets: History and context," ACM Trans. Interact. Intell. Syst., vol. 5, no. 4, Art. 19, Dec. 2015, doi: 10.1145/2827872.

[4] P. Lops, M. de Gemmis and G. Semeraro, "Content-based recommender systems: State of the art and trends," in Recommender Systems Handbook, F. Ricci, L. Rokach, B. Shapira and P. B. Kantor, Eds. New York, NY, USA: Springer, 2011, pp. 73–105, doi: 10.1007/978-0-387-85820-3_3.

[5] O. Barkan and N. Koenigstein, "Item2Vec: Neural item embedding for collaborative filtering," in Proc. IEEE 26th Int. Workshop on Machine Learning for Signal Processing (MLSP), Salerno, Italy, 2016. arXiv:1603.04259.

[6] X. He, L. Liao, H. Zhang, L. Nie, X. Hu and T.-S. Chua, "Neural collaborative filtering," in Proc. 26th Int. Conf. World Wide Web (WWW), Perth, Australia, 2017, pp. 173–182, doi: 10.1145/3038912.3052569.

[7] T. Hasan and R. Bunescu, "A survey of affective recommender systems: Modeling attitudes, emotions, and moods for personalization," arXiv:2508.20289 [cs.IR], Aug. 2025.

[8] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," ACM Trans. Inf. Syst., vol. 20, no. 4, pp. 422–446, Oct. 2002, doi: 10.1145/582415.582418.

[9] M. Kaminskas and D. Bridge, "Diversity, serendipity, novelty, and coverage: A survey and empirical analysis of beyond-accuracy objectives in recommender systems," ACM Trans. Interact. Intell. Syst., vol. 7, no. 1, 2016, doi: 10.1145/2926720.

[10] T. Mikolov, W.-t. Yih and G. Zweig, "Linguistic regularities in continuous space word representations," in Proc. 2013 Conf. North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Atlanta, GA, USA, 2013, pp. 746–751.

[11] T. Mikolov, K. Chen, G. Corrado and J. Dean, "Efficient estimation of word representations in vector space," arXiv:1301.3781, 2013.

[12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[13] Pallets Projects, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com

[14] The Movie Database (TMDB), "TMDB API documentation." [Online]. Available: https://www.themoviedb.org

Patent / Paper Publication Proof

[Insert proof of paper submission or publication here, if applicable. Remove this page otherwise.]

GLOSSARY OF TERMS

Embedding: A list of numbers (here 384) that represents the meaning of a text so that similar meanings are close together.

Sentence Transformer: A neural network that maps a sentence or paragraph to a single embedding vector.

Cosine similarity: A measure of how closely two vectors point in the same direction; 1 means identical direction.

Centroid: The mean of a set of vectors; here, the average of liked-movie embeddings, used as the user's taste.

Mood direction: A unit vector from the average movie toward the average of a mood's exemplar movies.

λ (lambda): The weight, between 0 and 1, of the mood direction when it is added to the taste vector.

TF-IDF: A weighting of words by frequency in a document and rarity across documents; ignores word meaning.

Cold start: The situation in which a new user or item has no history to base a recommendation on.

Collaborative filtering: Recommending items that people with similar likes enjoyed, using ratings rather than item descriptions.

Re-ranking: Reordering a shortlist of candidates with a second score; here the mood reorders the 60 best titles.

Hit Rate@K: The fraction of users for whom at least one relevant item appears in the top K.

NDCG@K: A ranking metric that rewards relevant items placed higher in the top K list.

Diversity: How different the recommended items are from each other.

Serendipity: How often relevant recommendations are also unexpected for the user.